

Sistemi Operativi (Modulo II)

Capitoli 0-1-2 — Richiami di architettura sistema operativo

Luca Argentino

A.A. 2025–2026

Indice

0	Introduzione ai Sistemi Operativi	3
0.1	Definizione e Obiettivi	3
0.1.1	Il S.O. come Macchina Estesa	3
0.1.2	Servizi Estesi Offerti dal S.O.	4
0.2	Evoluzione Storica	5
0.2.1	Generazione 0: Macchine Meccaniche	5
0.2.2	Generazione 0.5 (Anni '30-'40): Relè Elettromeccanici	5
0.2.3	Generazione 1 (1944–1955): Valvole e Tavole di Commutazione	5
0.2.4	Generazione 2 (1955–1965): Transistor e Sistemi Batch	6
0.2.5	Generazione 3 (1965–1980): Circuiti Integrati	7
0.2.6	Generazione 4 (1980–oggi): Personal Computer	9
0.3	Tipologie di Sistemi	10
0.3.1	Sistemi Paralleli	10
0.3.2	Sistemi Distribuiti	11
0.3.3	Sistemi Real-Time	11
1	Richiami di Architettura Hardware / Software	12
1.1	Gli Interrupt	13
1.1.1	Interrupt Hardware vs Trap (Software)	13
1.1.2	Il Ciclo di Gestione dell'Interrupt	14
1.1.3	Gestione degli Interrupt Multipli	15
1.2	Comunicazione tra Processore e Dispositivi di I/O	17
1.2.1	Programmed I/O (Obsoleto)	17
1.2.2	Interrupt-Driven I/O	17
1.2.3	Direct Memory Access (DMA)	17
1.2.4	Memory Mapped I/O	18
1.3	La Gerarchia di Memoria e il Caching	20
1.3.1	I Livelli della Gerarchia	20
1.3.2	Il Concetto di Caching	21
1.4	Meccanismi di Protezione Hardware	22
1.4.1	Doppia Modalità (User Mode / Kernel Mode)	22
1.4.2	Protezione dell'I/O e System Calls	22
1.4.3	Protezione della Memoria (MMU)	23
2	Architettura dei Sistemi Operativi	24
2.1	Introduzione e Componenti del Sistema	24
2.2	Struttura dei Sistemi Operativi	25
2.2.1	Sistemi con Struttura Semplice (Monolitici non strutturati)	26
2.2.2	Sistemi con Struttura a Strati (Layered)	27

2.3	Politiche e Meccanismi	31
2.4	Organizzazione del Kernel	31
2.4.1	Kernel Monolitici	31
2.4.2	Microkernel (Client/Server)	32
2.5	Confronto: Monolitico vs Microkernel	35
2.5.1	Architetture Alternative	35
2.6	Virtualizzazione e Isolamento	37
2.6.1	Macchine Virtuali (VM)	37
2.7	Namespace e Container (Virtualizzazione a Livello S.O.)	43
2.7.1	Il Problema: Perché Isolare i Processi?	43
2.7.2	I Tipi di Namespace in Linux	43
2.7.3	Le System Call dei Namespace	45
2.7.4	Cgroups: Limitazione delle Risorse	46
2.7.5	Container: Namespace + Cgroups	46
2.7.6	Confronto: Macchine Virtuali vs Container	47
2.7.7	Docker: il Runtime di Container (EXTRA)	48
2.8	Progettazione e System Generation	51
2.8.1	Parametri di Configurazione	51
2.8.2	Metodi di Generazione	51

Capitolo 0

Introduzione ai Sistemi Operativi

0.1 Definizione e Obiettivi

Definizione – Sistema Operativo

Il sistema operativo è un livello di astrazione che:

- Realizza il concetto di **processo**, creando l'astrazione processo e i meccanismi di sicurezza.
- Agisce come intermediario tra le applicazioni e l'hardware del calcolatore, controllandone l'esecuzione e mettendo a disposizione un “linguaggio” basato sulle *System Call*.

I suoi obiettivi principali sono:

- **Efficienza**: utilizzo ottimale delle risorse hardware dell'elaboratore.
- **Semplicità**: facilitare l'uso del computer per l'utente finale e per i programmatori.
- **Macchina Estesa**: nascondere i dettagli complessi dell'hardware fornendo una comoda API.
- **Portabilità**: consentire ai programmi di funzionare su architetture hardware diverse senza modifiche.

Nota – Il S.O. come Gestore di Risorse

A differenza dei normali sistemi di controllo (come un termostato, che è esterno al sistema controllato), il S.O. è esso stesso un programma. Per funzionare, deve cedere il controllo della CPU alle applicazioni e affidarsi a meccanismi hardware (come gli *interrupt*) per riottenere la gestione della macchina.

0.1.1 Il S.O. come Macchina Estesa

La visione “a strati” di un elaboratore moderno separa nettamente le responsabilità:

In questa visione, il S.O. nasconde ai programmatori i dettagli dell'hardware e fornisce una API conveniente, agendo da intermediario. Le parole chiave sono: **indipendenza dall'hardware, comodità d'uso e programmabilità**.

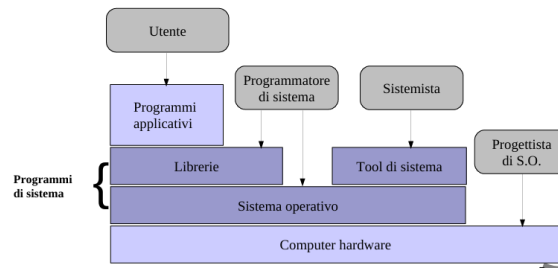


Figura 1

Esempio – Il S.O. come Macchina Estesa – Floppy Disk

Gestire un floppy disk a livello hardware (controllore NEC PD765, compatibile con le macchine Intel) richiede 16 comandi distinti (inizializzazione, avviamento motore, spostamento testina, lettura/scrittura...), con parametri impacchettati in fino a 9 byte e un output di 23 campi di stato in 7 byte. Il S.O. astrae tutto questo offrendo primitive semplici:

```
fd = open("/etc/rpc");
read(fd, buffer, size);
```

0.1.2 Servizi Estesi Offerti dal S.O.

Il S.O. espone le sue funzionalità attraverso le System Call, raggruppabili nelle seguenti categorie (già studiate nel I semestre):

- **Esecuzione di programmi:** caricamento ed avvio di processi.
- **Accesso semplificato/unificato ai dispositivi di I/O:** astrae le differenze tra periferiche.
- **Accesso al file system:** operazioni su file e directory.
- **Accesso al networking:** comunicazione in rete.
- **Accesso al sistema:** informazioni e controllo sullo stato del sistema.
- **Rilevazione e risposta agli errori:** gestione delle eccezioni hardware e software.
- **Accounting:** monitoraggio dell'uso delle risorse.

0.2 Evoluzione Storica

L'evoluzione dei S.O. è stata guidata dal progresso tecnologico nel campo dell'hardware, ma a sua volta ha *guidato* tale progresso: concetti come la gestione degli interrupt, la protezione della memoria e la memoria virtuale sono nati come esigenze software prima di diventare soluzioni hardware.

Nota – Approccio didattico

Il corso illustra ogni argomento partendo dalle prime soluzioni disponibili, costruendo sopra di esse soluzioni via via più complesse. Non stupirsi quindi se alcune soluzioni sembreranno banali o ingenue: sono state adottate 20, 30, 40 o 50 anni fa!

0.2.1 Generazione 0: Macchine Meccaniche

I primissimi tentativi di calcolo automatico risalgono al lavoro di Charles Babbage (1792–1871):

- **La Macchina Analitica:** Babbage tentò di costruire un calcolatore programmabile, di natura interamente meccanica.
- **Assenza di S.O.:** In questo periodo non esisteva alcun sistema operativo.
- **Curiosità storica:** La prima programmatrice della storia è Lady Ada Lovelace (figlia del poeta Lord Byron), che scrisse algoritmi per la macchina di Babbage.

0.2.2 Generazione 0.5 (Anni '30–'40): Relè Elettromeccanici

Prima dell'elettronica vera e propria, un periodo di transizione fu caratterizzato dai calcolatori elettromeccanici:

- **Macchine storiche:** lo Z3 di Konrad Zuse (1941) e l'Harvard Mark I (1944) di Howard Aiken.
- **Assenza di S.O.:** le istruzioni venivano lette sequenzialmente da nastri di carta perforata.
- **Caratteristiche:** più veloci dei meccanismi precedenti, ma ancora enormi, rumorose (il “clack” dei relè) e lentissime rispetto alle valvole successive.
- **Il primo “Bug”:** nel 1947, Grace Hopper trovò una falena reale incastrata nei contatti di un relè del Mark II. Da qui si popolarizzò il termine *debugging*.

0.2.3 Generazione 1 (1944–1955): Valvole e Tavole di Commutazione

- **Assenza di S.O. e Assembler:** la programmazione avveniva inserendo cavi nelle tavole di commutazione (linguaggio macchina puro). Non esisteva il concetto di assembler.
- **Ruoli sovrapposti:** progettista, costruttore, programmatore e operatore erano spesso la stessa persona; solo calcoli numerici (calcolatori, non elaboratori).
- **Problemi:** grossi limiti di affidabilità (guasti continui delle valvole), interazione lenta e complessa (caricamento manuale, attesa, reset).

0.2.4 Generazione 2 (1955–1965): Transistor e Sistemi Batch

L'introduzione dei transistor aumenta l'affidabilità e abbassa i costi, creando un vero mercato. Si ha una netta separazione dei ruoli: chi costruisce, chi opera e chi programma (in Fortran o Assembly su schede perforate).

Definizione – Job

Un programma o insieme di programmi la cui esecuzione veniva richiesta da uno degli utilizzatori. Era descritto da schede perforate con il programma, i dati di input e le istruzioni di controllo scritte in **JCL** (Job Control Language).

Definizione – Sistemi Batch (a lotti)

I primi rudimentali sistemi operativi erano detti **monitor residenti**: il controllo iniziale è nel monitor, che cede il controllo al job corrente e, al termine, lo riprende per caricare il successivo. Esegue *un solo job alla volta*.

• Memory layout per un S.O. batch (versione semplificata)

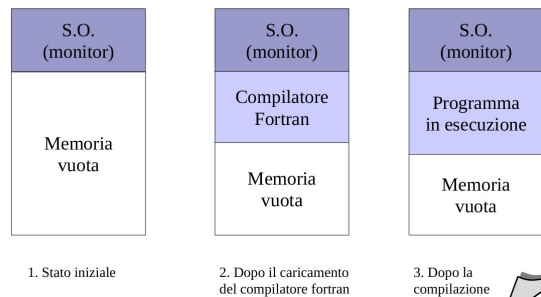


Figura 2: Job singolo caricato in memoria

Il ciclo di esecuzione di un job era il seguente: il programmatore perforava le schede e le consegnava all'operatore; l'operatore inseriva le schede di controllo JCL e le schede del programma; il monitor eseguiva il job e restituiva l'output. Si noti che **operatore** $\neq$ **programmatore**.

Memory layout semplificato di un sistema batch. L'evoluzione della memoria durante l'esecuzione di un job Fortran era la seguente:

1. **Stato iniziale:** solo il monitor occupa la memoria, il resto è vuoto.
2. **Caricamento compilatore:** il compilatore Fortran viene caricato in memoria assieme al monitor.
3. **Dopo la compilazione:** il compilatore viene rimosso e il programma compilato viene caricato per l'esecuzione.

Nota – Problemi di Efficienza

Nei sistemi batch la CPU restava inattiva (*idle*) durante le lente operazioni di I/O (lettura schede, stampa). Una prima parziale soluzione fu l'uso di nastri magnetici elaborati *off-line*: i job venivano caricati su nastro di input, elaborati con output su nastro, e solo in un secondo momento il nastro di output veniva stampato – su una macchina separata (e meno costosa). La macchina di calcolo principale (es. IBM 7094) restava così più libera per l'elaborazione.

0.2.5 Generazione 3 (1965–1980): Circuiti Integrati

L'hardware fa un salto di qualità con i circuiti integrati. Sparisce la figura dell'operatore come "interfaccia" verso la macchina: utente e operatore coincidono. I linguaggi usati sono C e shell scripting; si accede al sistema da terminali con editor testuali e grafici.

Definizione – Multiprogrammazione

Più job vengono caricati *contemporaneamente* in memoria. Per sfruttare i tempi morti di I/O, quando il job corrente richiede un'operazione di I/O, il **CPU scheduler** assegna la CPU a un altro job pronto. La CPU non viene mai lasciata *idle*.

• Caratteristiche tecniche:

- Più job contemporaneamente in memoria
- Una componente del S.O. detto scheduler si preoccupa di alternarli nell'uso della CPU
- quando un job richiede un'operazione di I/O, la CPU viene assegnata ad un altro job.

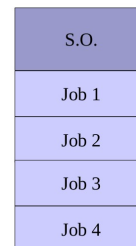


Figura 3: Job multipli caricati in memoria

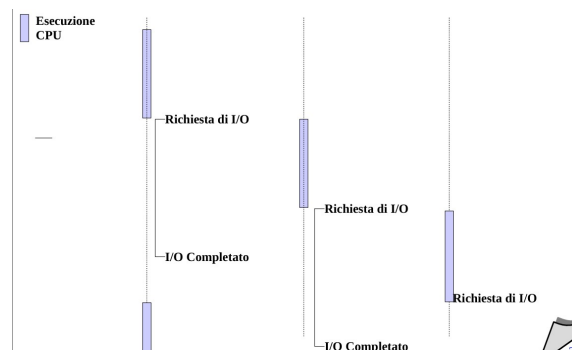


Figura 4: Funzionamento della multiprogrammazione

Un S.O. multiprogrammato deve fornire:

- **Routine di I/O:** gestite dal S.O. per conto dei job.
- **Gestione avanzata della memoria:** allocare la memoria per più job contemporanei.
- **CPU scheduling:** scegliere tra i diversi job pronti ad eseguire.
- **Allocazione delle risorse di I/O:** gestire i conflitti di accesso alle periferiche.

Definizione – Time-sharing

Estensione logica della multiprogrammazione per supportare l'interattività. Il tempo di CPU viene diviso in piccoli **quantità temporali**; allo scadere del timer hardware (*interval timer*), si forza il cambio di processo (*context switch*) anche in assenza di richieste di I/O. I context switch avvengono così frequentemente che più utenti possono interagire simultaneamente con i propri programmi.

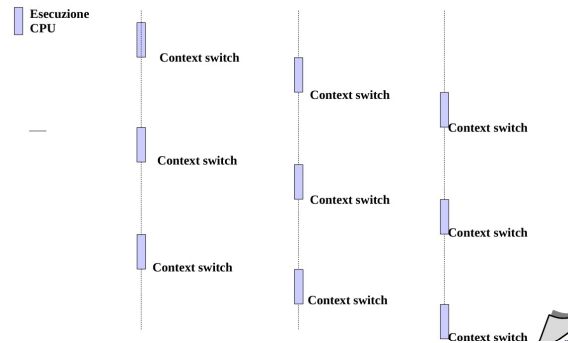


Figura 5: Funzionamento del Time sharing

Un S.O. time-sharing necessita di:

- **Memoria virtuale:** il numero di utenti e programmi può essere molto elevato.
- **Scheduling preemptive** (o time-sliced): sospensione periodica forzata dei processi.
- **Meccanismi di protezione:** protezione nel file system, nella memoria, tra utenti diversi.

Esempio – Nascita di UNIX e dei suoi predecessori

- **CTSS (1962):** Compatible Time-Sharing System. Introdusse i concetti di multiprogrammazione e time-sharing. Il codice sorgente è stato rilasciato nel settembre 2004.
- **Multics (1965):** Introdusse formalmente il concetto di processo.
- **UNIX (1970):** Derivato da CTSS e Multics, sviluppato ai Bell Labs inizialmente su PDP-7 e poi portato su PDP-11 – la prima volta che un S.O. veniva usato su due architetture diverse. Fu anche il primo S.O. riscritto quasi interamente in un linguaggio ad alto livello (C), rendendolo portabile. Dal ramo dei Bell Labs nacquero due grandi filoni: Bell Labs (SVR4, Solaris, AIX...) e **BSD** (Berkeley Software Distribution, da cui derivano FreeBSD, NetBSD, OpenBSD e MacOS X).

0.2.6 Generazione 4 (1980–oggi): Personal Computer

L'avvento dei microprocessori porta i computer nelle case e negli uffici di singoli utenti.

Esempio – Frase celebre

“Non c'è ragione per cui qualcuno possa volere un computer a casa sua” (Ken Olson, fondatore di Digital Equipment Corporation, 1977)

- I personal computer sono dedicati a **singoli utenti**: l'obiettivo primario diventa la **facilità d'uso** (nascita delle GUI, che però non fanno strettamente parte del kernel del S.O.).
- Essendo dedicati a un singolo utente, i S.O. per PC sono in generale più semplici rispetto ai predecessori.
- **Grave errore storico**: la sicurezza è stata gravemente sottovalutata, un errore pagato a caro prezzo con l'avvento di Internet.

0.3 Tipologie di Sistemi

I sistemi informatici moderni si estendono ben oltre la singola macchina isolata.

0.3.1 Sistemi Paralleli

Definizione – Sistema Parallelo

Un singolo elaboratore che possiede più unità di elaborazione (CPU) strettamente interconnesse.

I sistemi paralleli si classificano secondo tre tassonomie indipendenti:

Tassonomia basata sulla struttura:

- **SIMD** (Single Instruction, Multiple Data): tutte le CPU eseguono all'unisono lo stesso programma su dati diversi. Tipico delle schede grafiche (GPU).
- **MIMD** (Multiple Instruction, Multiple Data): le CPU eseguono programmi differenti su dati differenti. Tipico dei processori multicore.

Tassonomia basata sulla dimensione:

- **Sistemi a basso parallelismo**: pochi processori, in genere molto potenti.
- **Sistemi massicciamente paralleli**: gran numero di processori, che possono anche avere potenza non elevata individualmente.

Tassonomia basata sull'accoppiamento:

- **Tightly coupled** (strettamente accoppiati): bus e memoria condivisa. Pochi processori, basso livello di parallelismo; il bus può diventare un collo di bottiglia.
- **Loosely coupled** (debolmente accoppiati): ogni processore ha memoria privata e comunicano tramite canali. Tanti processori, alto livello di parallelismo.

Il vantaggio principale dei sistemi paralleli è l'**incremento delle prestazioni** grazie all'esecuzione simultanea di più operazioni.

Organizzazione del S.O.:

- **SMP (Symmetric Multiprocessing)**: ogni processore esegue una copia identica del S.O. in modo paritetico. Processi diversi possono girare contemporaneamente su CPU diverse. Supportato dalla maggior parte dei S.O. moderni.
- **Asymmetric Multiprocessing**: ogni processore è assegnato a un compito specifico; un processore *master* gestisce l'allocazione del lavoro ai processori *slave*. Più comune in sistemi estremamente grandi.

0.3.2 Sistemi Distribuiti

Definizione – Sistema Distribuito

Insieme di elaboratori indipendenti (con proprie risorse e proprio S.O.) collegati in rete, che appaiono all'utente come se fossero un unico sistema coerente.

I vantaggi principali sono: condivisione di risorse, suddivisione del carico (load balancing), incremento delle prestazioni e alta affidabilità (tolleranza ai guasti). La latenza è tuttavia maggiore rispetto ai sistemi paralleli.

Nota – S.O. in rete vs S.O. Distribuiti

È importante distinguere due livelli di integrazione:

- **S.O. in rete:** ogni computer opera in modo largamente indipendente dagli altri; il S.O. si limita a fornire condivisione di file e possibilità di comunicare.
- **S.O. distribuiti:** minore autonomia tra i computer; il S.O. dà l'impressione che un singolo sistema operativo stia controllando tutti gli elaboratori del sistema. L'utente non è consapevole di quale macchina stia eseguendo il suo processo.

0.3.3 Sistemi Real-Time

Definizione – Sistema Real-Time

Un sistema in cui la correttezza del risultato dipende *non solo* dal suo valore, ma anche dall'**istante temporale** in cui il risultato viene prodotto. Real-time **non significa** necessariamente “veloce”, ma **predicibile e puntuale**.

- **Hard Real-Time:** il mancato rispetto delle scadenze (*deadline*) provoca effetti catastrofici. Esempi: controllo assetto velivoli, controllo centrali nucleari, apparecchiature per terapia intensiva.
- **Soft Real-Time:** il ritardo causa solo disagi o disservizi. Esempi: programmi interattivi, streaming multimediale.

Capitolo 1

Richiami di Architettura Hardware / Software

Per comprendere a fondo il funzionamento di un sistema operativo, è essenziale richiamare i concetti base dell'architettura hardware/software su cui esso opera, basata sul modello di Von Neumann. L'hardware fornisce i meccanismi fisici che il S.O. utilizza per gestire le risorse e garantire l'isolamento e la protezione dei processi.

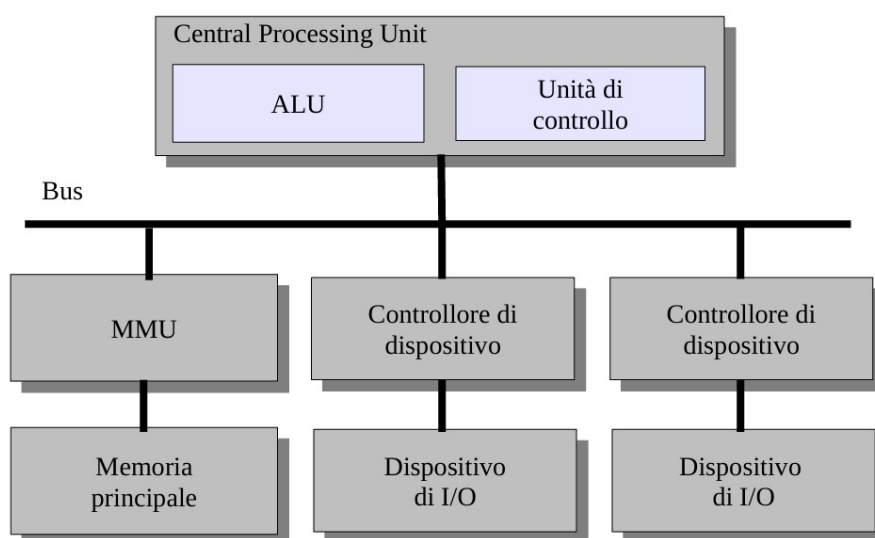


Figura 1.1: **Architettura di Von Neumann.** Il sistema è strutturato attorno a un **Bus** di comunicazione centrale. La **Central Processing Unit (CPU)** è partizionata logicamente in 2 sotto-unità: l'**ALU** (Arithmetic Logic Unit), l'**Unità di controllo**. Al bus sono collegati direttamente la **Memory management unit (MMU)** e i vari **Controllori di dispositivo**, i quali fungono da intermediari hardware per governare i rispettivi **Dispositivi di I/O** periferici.

Nota – Hardware vs Software nella gestione dei dispositivi

Spesso si crea confusione tra le componenti fisiche e i moduli software che le governano. È fondamentale distinguere:

- **Device (HW)**: Il dispositivo fisico vero e proprio (es. disco magnetico, tastiera).
- **Device Controller (HW)**: La scheda elettronica che controlla fisicamente il dispositivo e comunica con il bus di sistema.
- **Device Driver (SW)**: Il codice fornito al S.O. per "parlare" con il controller specifico.
- **Memory Management Unit - MMU (HW)**: L'unità circuitale che si occupa di tradurre gli indirizzi e proteggere la memoria.
- **Memory Manager (SW)**: Il modulo del S.O. che configura e controlla le funzionalità della MMU.

1.1 Gli Interrupt

I sistemi operativi moderni sono definiti "**Interrupt Driven**". Questo significa che il codice del S.O. non è costantemente in esecuzione, ma entra in funzione (spesso come *interrupt handler*) solo in risposta a specifici eventi che ne richiedono l'intervento per avvicinare i processi o gestire l'hardware.

Definizione – Interrupt

Un meccanismo (hardware o software) che permette di interrompere il normale ciclo di esecuzione (fetch-execute) della CPU per gestire tempestivamente eventi. Sono stati introdotti per aumentare l'efficienza del calcolatore, permettendo al S.O. di intervenire solo quando necessario.

1.1.1 Interrupt Hardware vs Trap (Software)

Gli interrupt (ovvero i segnali mandati ai processori) si dividono in due macro-categorie in base alla loro origine di generazione:

- **Interrupt Hardware**: Sono eventi asincroni generati dai controller dei dispositivi, non dipendenti dall'istruzione che la CPU sta eseguendo in quel momento, per far capire al processore che sono avvenuti eventi che necessitano di attenzione.
Esempi tipici sono la notifica di completamento di un'operazione di I/O o la scadenza del *clock/interval timer* (fondamentale per il time-sharing).
- **Interrupt Software (Trap)**: Sono eventi sincroni causati volontariamente o involontariamente dal processo in esecuzione.
Includono errori a tempo di esecuzione (come divisione per zero o accesso ad area di memoria non consentita) o la richiesta esplicita di servizi al S.O. tramite *System Call*.

1.1.2 Il Ciclo di Gestione dell'Interrupt

Quando un controller solleva un interrupt, si innesca una precisa sequenza di azioni che coinvolge sia l'hardware che il software:

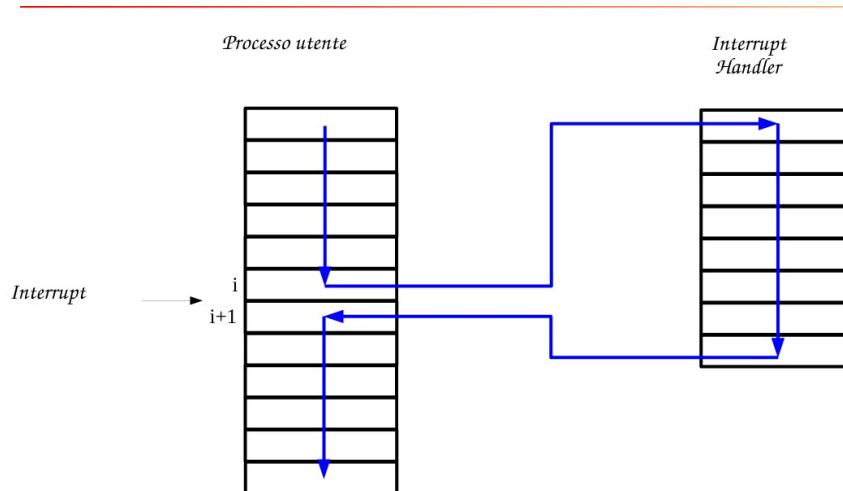


Figura 1.2

Nella gestione di un interrupt, le responsabilità sono nettamente divise tra i meccanismi fisici della CPU e il codice del Sistema Operativo.

Azione del Processore (Hardware):

1. **Richiesta e Rilevazione:** Un segnale di "interrupt request" viene spedito alla CPU. La CPU completa l'esecuzione dell'istruzione corrente e verifica la presenza del segnale.
2. **Salvataggio Critico (Hardware):** La CPU prepara il trasferimento di controllo salvando in automatico lo stato minimo indispensabile per non perdere il filo dell'esecuzione. Questo avviene tramite due possibili approcci:
 - *Salvataggio dei registri critici:* si memorizzano informazioni essenziali come il Program Counter (PC) e il registro di stato.
 - *Scambio di stato:* si salva una "fotografia" dell'intero stato del processore.
3. **Selezione e Salto all'Handler:** La CPU individua l'Interrupt Handler appropriato tramite l'*Interrupt Vector* (a seconda dell'architettura, ci può essere un singolo handler, uno per tipo di interrupt, o uno per dispositivo). A questo punto, **l'hardware sovrascrive il PC** con l'indirizzo iniziale dell'handler, effettuando il salto.

Nota – Transizione HW/SW

Tutte le operazioni fino al caricamento del nuovo PC sono puramente hardware. Solo da questo momento il ciclo fetch-execute riprende e il controllo passa effettivamente al software (S.O.).

Azione del Sistema Operativo (Software):

1. **Salvataggio dello Stato Aggiuntivo:** Una volta in esecuzione, l'handler si occupa di salvare nei propri registri o nello stack il resto dello stato del

processore (i registri di uso generale) che non è stato salvato automaticamente dall'hardware.

2. **Gestione dell'evento:** Viene eseguita la logica necessaria. Il S.O. legge le informazioni di controllo provenienti dal dispositivo (es. fine I/O) ed eventualmente spedisce ulteriori comandi.
3. **Ripristino e Ritorno:** Avviene l'operazione inversa del salvataggio software: viene ripristinato lo stato del processore e, tramite un'istruzione speciale (come `rfe/eret`), si restituisce il controllo al processo interrotto, oppure a un altro processo qualora sia intervenuto lo scheduler.

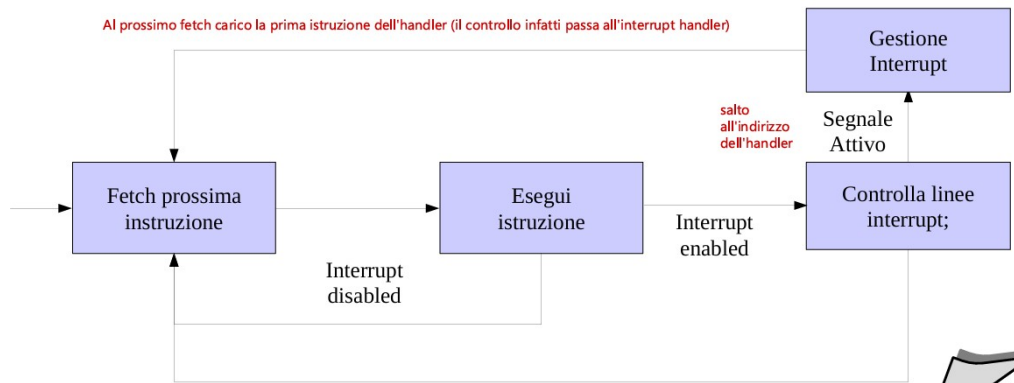


Figura 1.3

1.1.3 Gestione degli Interrupt Multipli

Cosa succede se si verifica un interrupt mentre il processore è già all'interno di un interrupt handler? Vi sono due approcci principali per gestire queste sovrapposizioni:

- **Disabilitazione degli Interrupt:** Durante la gestione di un interrupt, i successivi vengono temporaneamente ignorati (mascherati) o lasciati "pendenti". Verranno riabilitati solo prima di tornare al processo interrotto. È un approccio semplice, che garantisce un'esecuzione sequenziale, ma risulta inadatto per sistemi con vincoli *time-critical*.

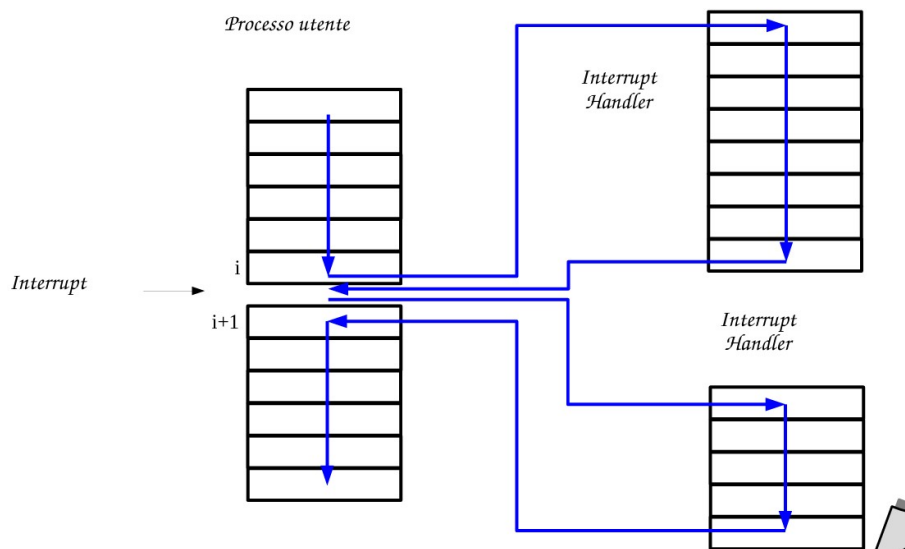


Figura 1.4

- **Interrupt Annidati:** Ai vari interrupt vengono assegnate delle *priorità*. Se arriva un interrupt con priorità superiore rispetto a quello in gestione, il S.O. sospende l'handler corrente per servire quello nuovo. Questo approccio è più complesso e richiede l'uso di stack separati, ma è fondamentale per servire dispositivi veloci (come le schede di rete) che non possono aspettare.

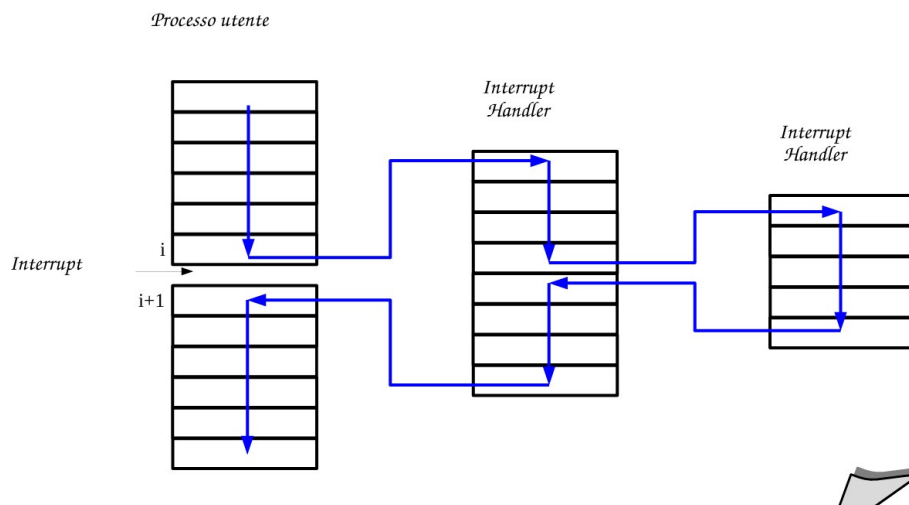


Figura 1.5

1.2 Comunicazione tra Processore e Dispositivi di I/O

I dispositivi sono molto più lenti della CPU. Di conseguenza, nel corso della storia sono state adottate diverse modalità per gestire il trasferimento dati senza penalizzare le prestazioni dell'intero sistema.

1.2.1 Programmed I/O (Obsoleto)

In questa modalità primitiva, la CPU carica (tramite bus) i parametri della richiesta di I/O nei registri del controller e attende attivamente (*polling* o *busy waiting*) finché l'operazione non è terminata.

Nota – Svantaggio del Programmed I/O

Il processore spreca una quantità enorme di tempo interrogando ripetutamente lo status register del controller. La velocità di trasferimento dipende totalmente dal processore, annullando di fatto la possibilità di fare multiprogrammazione.

1.2.2 Interrupt-Driven I/O

Con questa tecnica, dopo aver inviato il comando di I/O al controller, il S.O. sospende il processo richiedente e assegna la CPU ad un altro processo pronto. Il controller esegue fisicamente la richiesta posizionando i dati nel suo buffer interno. Una volta terminato, genera un interrupt. A questo punto la CPU interrompe ciò che stava facendo e copia i dati dal buffer locale alla memoria centrale. Si elimina l'attesa in idle, ma il processore deve ancora occuparsi fisicamente del trasferimento dati.

1.2.3 Direct Memory Access (DMA)

Definizione – Direct Memory Access (DMA)

È una tecnica hardware che delega il lavoro di trasferimento dati direttamente al controller. Il S.O. imposta l'operazione fornendo al controller l'indirizzo di memoria sorgente o destinazione. A questo punto, il dispositivo accede **direttamente** alla memoria centrale scavalcando la CPU, prelevando o scrivendo i dati in autonomia.

L'interrupt viene generato *solo* al termine dell'intero blocco di dati trasferito.

- **Vantaggi:** Altamente efficiente. La CPU non deve occupare i propri cicli di clock per spostare byte tra controller e memoria. I device driver risultano anche semplificati.
- **Svantaggi:** Essendo il bus di sistema condiviso, può verificarsi una contesa (*bus contention*) tra la CPU e il DMA per l'accesso simultaneo alla memoria.

1.2.4 Memory Mapped I/O

Definizione – Memory Mapped I/O

In questa architettura, un dispositivo è completamente indirizzabile tramite bus. I dati gestiti dal dispositivo vengono mappati su un insieme di indirizzi direttamente accessibili tramite il bus di sistema. Di conseguenza, una normale operazione di lettura o scrittura su questi specifici indirizzi causa direttamente il trasferimento di dati da o verso il dispositivo stesso.

Esempio – Esempio Tipico

Un classico esempio di utilizzo del Memory Mapped I/O è la gestione del **video grafico nei PC**. Quando il sistema interagisce con quegli indirizzi, sta comunicando direttamente con l'hardware per l'output a schermo (come avviene, ad esempio, sui nostri monitor Signal Urgent).

Nota – Vantaggi e Svantaggi

L'utilizzo di questa tecnica si riassume in due punti chiave:

- **Vantaggi:** La gestione risulta **molto semplice e lineare**.
- **Svantaggi:** Sfruttando il normale bus di sistema, questa architettura **richiede di tecniche di sincronizzazione di accesso**.

Differenza tra I/O Isolato (Normale) e Memory Mapped I/O

Per comprendere appieno il Memory Mapped I/O, è indispensabile confrontarlo con l'approccio tradizionale, noto come **I/O a porte isolate** (o *Port-Mapped I/O*). La distinzione principale risiede in come la CPU "vede" e gestisce la comunicazione con l'hardware.

• I/O a Porte Isolate (Approccio "Normale")

- **Spazi separati:** Esiste uno spazio di indirizzamento per la memoria RAM e uno spazio completamente distinto per le periferiche di I/O. L'indirizzo "100" della RAM è una cosa diversa dalla porta hardware "100".
- **Istruzioni dedicate:** La CPU possiede un set di istruzioni Assembly specifico (es. IN e OUT) che serve esclusivamente per parlare con i dispositivi hardware.
- **Segnali di controllo:** Sul bus di sistema è presente una linea di controllo aggiuntiva. La CPU la usa per segnalare se l'indirizzo che sta inviando sul bus è destinato alla memoria oppure a una periferica I/O.

• Memory Mapped I/O

- **Spazio unico:** Un dispositivo è completamente indirizzabile tramite bus; i suoi dati vengono mappati su un insieme di indirizzi direttamente accessibili e condivisi con la memoria principale.
- **Istruzioni standard:** Non esistono comandi speciali. La CPU usa le normali istruzioni di accesso alla memoria (come le istruzioni **LOAD** e **STORE**).
- **Gestione del bus:** Quando la CPU esegue una scrittura su quegli specifici indirizzi, la RAM li ignora, ma il dispositivo periferico li riconosce e intercetta il dato in transito sul bus.

Nota – Tabella Riassuntiva di Confronto

Caratteristica	I/O Isolato (Port-Mapped)	Memory Mapped I/O
Spazio Indirizzi	Separato (RAM $\neq$ Periferiche)	Unico e Condiviso
Istruzioni CPU	Dedicate (IN, OUT)	Standard (LOAD, STORE)
Segnali sul Bus	Richiede pin/segnali aggiuntivi	Usa il bus standard senza modifiche
Impatto sulla RAM	Tutto lo spazio indirizzi è per la RAM	Sottrae indirizzi utilizzabili dalla RAM

1.3 La Gerarchia di Memoria e il Caching

La memoria ideale per un elaboratore dovrebbe essere infinitamente grande, infinitamente veloce e molto economica. Essendo fisicamente impossibile avere tutte e tre le caratteristiche contemporaneamente, l'architettura dei calcolatori è strutturata su una **gerarchia di memorie** che bilancia questi *trade-off*.

Nota – Il Trade-off delle Memorie

La progettazione della memoria si scontra costantemente con limitazioni fisiche ed economiche:

- Un tempo di accesso più veloce comporta inevitabilmente un costo maggiore.
- Una maggiore capacità comporta un costo minore per bit, ma porta con sé un tempo di accesso maggiore.

1.3.1 I Livelli della Gerarchia

Procedendo dal processore verso l'esterno, la memoria si divide in *volatile* e *non volatile*, organizzandosi in memoria principale, secondaria e terziaria:

- **Memoria Principale (Volatile):**
 - **Registri e Cache CPU:** Memorie piccolissime ma estremamente veloci, integrate direttamente nel processore.
 - **Memoria Centrale (RAM):** Insieme ai registri, è l'unico spazio di memorizzazione a cui la CPU può accedere direttamente, operando tramite le istruzioni **LOAD** e **STORE**. L'accesso avviene tramite la MMU, che trasforma a tempo di esecuzione gli indirizzi logici in indirizzi fisici. È una memoria volatile.
- **Memoria Secondaria (Non Volatile):**
 - **Dischi Magnetici (HDD):** Dispositivi che offrono accesso diretto (random, ovvero non puramente sequenziale) ai dati. Per individuare fisicamente un dato sul disco, occorre indirizzarlo in termini di *cilindro*, *settore* e *testina*. Le operazioni gestite dal controller sono **READ**, **WRITE** e **SEEK**. L'operazione di **SEEK** (lo spostamento fisico del pettine delle testine da un cilindro all'altro) è di gran lunga la più costosa. Le operazioni di lettura e scrittura prevedono inoltre un'attesa affinché il disco ruoti fino a portare il settore richiesto sotto la testina.
 - **Unità a Stato Solido (SSD / SDcard):** Tecnologie per la memorizzazione non volatile prive di parti meccaniche. Si differenziano dai dischi tradizionali perché la lettura avviene a "blocchi", mentre la scrittura viene effettuata a "banchi" (che raggruppano numerosi blocchi). Hanno la particolarità di supportare un numero di cicli di scrittura limitato prima di deteriorarsi.
- **Memoria Terziaria (Non Volatile):** Destinata all'archiviazione di massa a lungo termine, include reti SAN (Storage Area Network) e unità off-line.

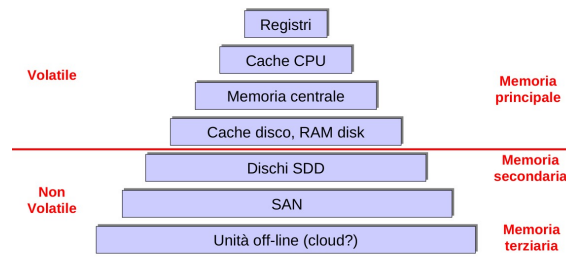


Figura 1.6

1.3.2 Il Concetto di Caching

Definizione – Il Principio del Caching

Un meccanismo di caching consiste nel memorizzare parzialmente i dati provenienti da una memoria lenta all'interno di una seconda memoria più costosa ma più efficiente. Se il numero di occorrenze in cui il dato ricercato viene trovato nella memoria veloce (nella *cache*) è statisticamente rilevante rispetto agli accessi totali, il sistema ottiene un notevole aumento di prestazione globale.

Il concetto di cache è trasversale e si applica a innumerevoli livelli dell'architettura:

- Cache della memoria principale (DRAM) realizzata tramite memoria veloce bipolare.
- Cache del disco mantenuta in memoria centrale (RAM disk).
- Cache di file system remoti salvata tramite file system locali.

Esempio – Gestione della Cache: Hardware vs Software

I meccanismi di caching possono essere implementati e gestiti a due livelli distinti:

- **Gestione Hardware:** Riguarda ad esempio le Cache L1/L2 interne alla CPU. Le politiche di gestione dei dati sono decise dall'hardware e non sono modificabili dal Sistema Operativo.
- **Gestione Software:** Riguarda ad esempio la cache del disco in memoria centrale. Le politiche sono interamente implementate in codice e sotto il pieno controllo del Sistema Operativo.

Nota – Problemi da considerare per il S.O.

L'implementazione software del caching obbliga il progettista del S.O. a risolvere due problemi critici:

- **Algoritmo di Replacement (Sostituzione):** Poiché la cache ha una dimensione fortemente limitata, il S.O. deve decidere costantemente quali vecchi dati scartare per far spazio ai nuovi. L'obiettivo dell'algoritmo è garantire il maggior numero possibile di accessi futuri con successo in cache.
- **Coerenza dei Dati:** Gli stessi identici dati possono apparire a diversi livelli della struttura gerarchica (es. un blocco di testo presente sia su disco che in cache RAM). Il S.O. deve garantire che le modifiche fatte

a un livello vengano sincronizzate correttamente con gli altri livelli, per evitare disallineamenti.

1.4 Meccanismi di Protezione Hardware

In un sistema operativo che supporta la multiprogrammazione e il time-sharing, i processi utente convivono nella stessa macchina. Per evitare che processi fallati o malintenzionati mandino in crash l'intero elaboratore interferendo con le risorse altrui o con lo stesso S.O., sono indispensabili **meccanismi di protezione forniti a livello hardware**.

1.4.1 Doppia Modalità (User Mode / Kernel Mode)

I processori implementano almeno due modalità di esecuzione, differenziate da un *Mode bit* presente nel registro di stato:

- **Modalità Kernel (o Supervisore / Ring 0):** In questa modalità, la CPU può eseguire *qualsiasi* istruzione del suo set, incluse le *istruzioni privilegiate* (es. disabilitare gli interrupt, gestire I/O diretti). Il Sistema Operativo gira in questa modalità.
- **Modalità Utente:** I processi non hanno accesso alle istruzioni privilegiate.

All'avvio (*bootstrap*) la macchina è in modalità Kernel per caricare il S.O. Quando lo scheduler passa l'esecuzione a un programma utente, il Mode bit viene switchato a Utente. **A ogni interrupt o trap**, l'hardware rimette autonomamente il Mode bit a Kernel prima di cedere il controllo all'Interrupt Handler.

1.4.2 Protezione dell'I/O e System Calls

Tutte le istruzioni che comunicano direttamente con i dispositivi hardware sono definite *privilegiate*, eseguite unicamente dal S.O. Se un processo utente potesse dialogare liberamente col disco, potrebbe aggirare i permessi del *file system* distruggendo i dati di altri utenti o del S.O..

Nota – Le System Call

Poiché l'utente non può usare istruzioni di I/O, il S.O. gli fornisce un'interfaccia protetta: le **System Call**. Eseguendo l'istruzione speciale `syscall`, il processo genera una Trap (interrupt software). La CPU commuta automaticamente in Kernel Mode e salta alla routine del S.O., il quale verifica i permessi, esegue l'operazione di I/O richiesta e infine restituisce il controllo (e i risultati) al processo, ripristinando la modalità utente tramite `rfe/eret`.

1.4.3 Protezione della Memoria (MMU)

Non basta proteggere i dispositivi: occorre impedire che un processo acceda alla porzione di memoria RAM assegnata ad un altro processo, al S.O. o che vada ad alterare l'*Interrupt Vector*.

Definizione – Memory Management Unit (MMU)

Componente hardware essenziale per la protezione della memoria. Ogni volta che un processo in esecuzione genera un indirizzo (Indirizzo Logico), questo non viene mandato direttamente al bus, ma passa attraverso la MMU.

La MMU controlla l'indirizzo logico per verificare che rientri nello spazio consentito a quel processo. Se è valido, lo traduce in Indirizzo Fisico. Se il programma tenta di generare un indirizzo non consentito, la MMU blocca l'accesso generando una Trap di "Memory Fault" e il S.O. interrompe in modo anomalo il programma trasgressore.

Capitolo 2

Architettura dei Sistemi Operativi

2.1 Introduzione e Componenti del Sistema

I sistemi operativi possono essere profondamente diversi nella loro organizzazione interna, e la progettazione della loro architettura rappresenta un problema ingegneristico fondamentale.

Un S.O. può essere osservato da diversi punti di vista: quello del **programmatore** (interfaccia di sistema), quello dell' **utente** (servizi forniti tramite API/System Call) e quello del **progettista** (struttura e componenti interne).

Le componenti logiche principali che costituiscono un sistema operativo sono:

- **Gestione dei Processi:** Un processo è un'attività di un programma in esecuzione che utilizza le risorse fornite dal computer per assolvere i propri compiti.
Compito S.O. si occupa della creazione, terminazione, sospensione e riattivazione dei processi, oltre a fornire meccanismi per la sincronizzazione, la comunicazione inter-processo (IPC) e la gestione dei *deadlock* (stalli).
- **Gestione della Memoria Principale:** La memoria principale (RAM) è un "array" di byte indirizzabili singolarmente. È un deposito di dati volatile, destinato a contenere i processi in esecuzione e i dati utilizzati, che deve essere facilmente e rapidamente accessibile dalla CPU (e dai dispositivi di I/O tramite DMA).
Compiti del S.O.:
 - **Tracking:** Tenere traccia istante per istante di quali parti della memoria sono in uso e da parte di chi (quale processo).
 - **Admission Control:** Decidere quali processi caricare in memoria quando si libera dello spazio.
 - **Allocazione:** Allocare e deallocare lo spazio di memoria in base alle richieste dei processi e alle politiche di sistema.
 - **Memoria Virtuale:** Gestire lo spostamento di dati tra memoria principale e secondaria per simulare una quantità di RAM maggiore di quella fisica.

- **Gestione della Memoria Secondaria:** Poiché la memoria principale è volatile e limitata, il computer necessita di una memoria secondaria (Hard Disk, SSD, Dischi Ottici) per memorizzare dati e programmi in modo permanente.

Compiti del S.O.:

- **Partizionamento:** Gestione dell’allocazione dello spazio libero e occupato sui dischi.
- **Disk Scheduling:** Ordinamento efficiente delle richieste di I/O per minimizzare i movimenti delle testine e migliorare le prestazioni.
- **Affidabilità e Prestazioni:** Gestione di meccanismi avanzati come il RAID per garantire accesso efficiente e tolleranza ai guasti.
- **Gestione del File System:** Un file è l’astrazione logica di un archivio di dati, definita in modo indipendente dalle caratteristiche fisiche del supporto su cui è memorizzato. Il File System è l’insieme organizzato di questi file.

Compiti del S.O.:

- **Gestione Elementi:** Fornire meccanismi per la creazione, cancellazione e manipolazione di file e directory.
- **Mapping:** Mappare l’astrazione logica (file e cartelle) sulla sequenza fisica di blocchi o settori del disco.
- **Backup:** Supportare operazioni per la salvaguardia dei dati su supporti non volatili.
- **Gestione dei Dispositivi di I/O:** Fornisce un’interfaccia comune per i *device driver*, gestisce i driver specifici per l’hardware e fornisce sistemi di buffer per il caching.
- **Supporto Multiuser e Protezione:** Meccanismi per controllare l’accesso di utenti e processi alle risorse (identità UID/GID e permessi).
- **Networking:** Protocolli di comunicazione a basso livello (IP, TCP, UDP) e servizi ad alto livello (file system distribuiti come NFS o SMB). consente di far comunicare processi in sequenza su più elaboratori e di condividere risorse

2.2 Struttura dei Sistemi Operativi

Definizione – Architettura di un sistema operativo

Descrive quali sono le varie componenti del S.O. e come queste sono collegate fra loro.

I vari sistemi operativi sono molto diversi l’uno dall’altro nella loro architettura

La progettazione di un S.O. deve bilanciare caratteristiche che spesso sono in antitesi (*trade-off*), come efficienza, manutenibilità, modularità ed espansibilità. In base alla loro evoluzione e organizzazione, i S.O. si dividono storicamente in **due grandi famiglie**:

- Sistemi con struttura semplice
- Sistemi con struttura a strati

2.2.1 Sistemi con Struttura Semplice (Monolitici non strutturati)

Si tratta di sistemi operativi nati inizialmente per hardware limitato e cresciuti nel tempo senza una reale progettazione architetturale a priori. Possono essere descritti come un'enorme collezione di procedure che condividono lo stesso spazio di indirizzamento, in cui ciascuna procedura può chiamare liberamente le altre senza vincoli di protezione o gerarchia.

Esempio – Esempi storici: MS-DOS e il primo UNIX

- **MS-DOS:** In questo sistema, le interfacce e i livelli di funzionalità non erano ben separati.
 - **Accesso Diretto:** Un programma applicativo (utente) poteva accedere direttamente alle routine del BIOS o ai driver hardware per fare I/O, scavalcando completamente il sistema operativo.
 - **Fragilità:** La mancanza di separazione (dovuta all'hardware dell'epoca, come l'Intel 8086, che non supportava la *Dual Mode User/Kernel*) faceva sì che un programma difettoso o maligno potesse sovrascrivere dati critici e mandare in crash l'intero sistema.
 - **Estensibilità (TSR):** Il sistema base era monotask. Tuttavia, utilizzava programmi speciali chiamati **TSR** (*Terminate and Stay Resident*): questi programmi, una volta eseguiti, rimanevano residenti in memoria per fornire servizi aggiuntivi o driver, ampliando di fatto l'interfaccia del sistema per gli altri programmi.
- **UNIX Originale:** Pur essendo più strutturato rispetto a MS-DOS, anche il primo UNIX presentava una struttura limitata, suddivisa grossolanamente in due parti separate dall'interfaccia delle *System Call*:
 - **Kernel:** Un unico blocco monolitico situato direttamente sopra l'hardware. Conteneva al suo interno tutte le funzioni critiche: *file system*, *CPU scheduling*, *gestione della memoria* e *driver di dispositivo*.
 - **Programmi di Sistema:** Tutto ciò che si trova sopra l'interfaccia delle System Call (shell, compilatori, interpreti).
 - Sebbene monolitico, UNIX introdusse una separazione logica più netta rispetto al DOS, definendo confini precisi tra l'hardware e il software utente. L'utente infatti non tocca l'hardware ma chiede al kernel di farlo tramite le SYS call

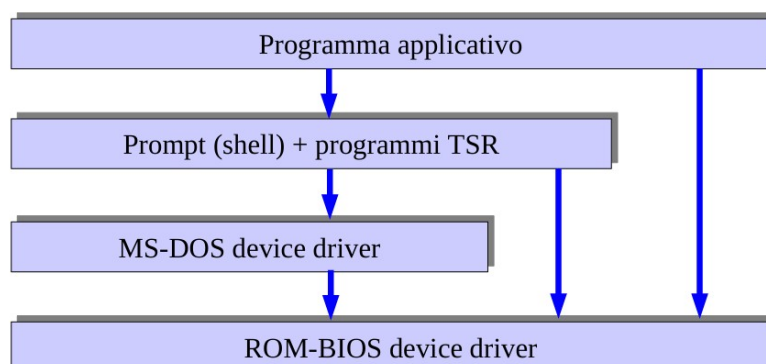


Figura 2.1: **Struttura di MS-DOS.** Si nota come il programma applicativo possa bypassare il S.O. e accedere direttamente ai driver del BIOS.

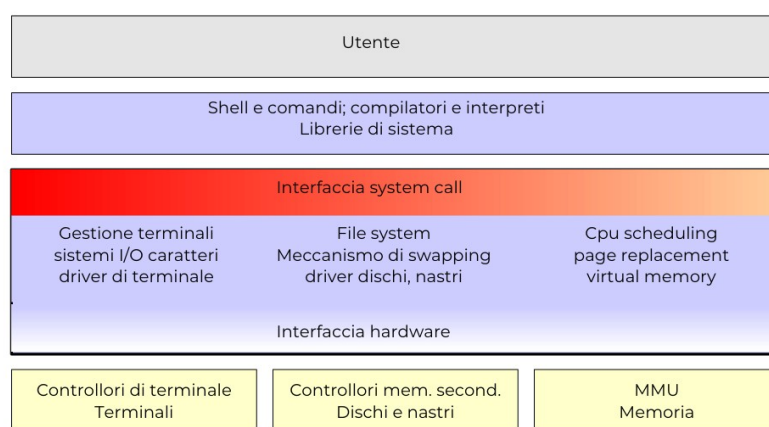


Figura 2.2: **Struttura di UNIX.** Il Kernel agisce come intermediario unico tra l'hardware sottostante e l'interfaccia delle System Call.

2.2.2 Sistemi con Struttura a Strati (Layered)

In questo approccio, il sistema operativo è suddiviso in un numero di livelli (layer), ciascuno costruito sopra quello inferiore. Il livello più basso (livello 0) è l'hardware, mentre quello più alto (livello N) è l'interfaccia utente.

Il principio fondamentale è che un livello M è costituito da un'implementazione di oggetti astratti (dati e operazioni) che:

- Utilizza **esclusivamente** i servizi offerti dal livello inferiore $M - 1$.
- Offre servizi al livello superiore $M + 1$, nascondendo i dettagli implementativi interni (*information hiding*).

- | | |
|--|--|
| <ul style="list-style-type: none"> • The O.S. (Dijkstra) 5) Programmi utente 4) Gestione I/O 3) Console device/driver 2) Memory management 1) CPU Scheduling 0) Hardware | <ul style="list-style-type: none"> • Venus OS 6) Programmi utente 5) Device driver e scheduler 4) Memoria virtuale 3) Canali di I/O 2) CPU Scheduling 1) Interprete di istruzioni 0) Hardware |
|--|--|

Figura 2.3: **Confronto tra Gerarchie a Strati.** A sinistra il sistema *THE* (6 livelli), dove lo scheduling della CPU è gestito quasi a livello hardware (Livello 1). A destra il sistema *Venus* (7 livelli), che introduce un livello intermedio di interpretazione istruzioni prima dello scheduling (Livello 2). In entrambi i casi, l'astrazione cresce dal basso (Hardware) verso l'alto (Utente).

Nota – Vantaggi e Svantaggi

- **Vantaggi (Modularità):** La struttura a strati semplifica drasticamente il debugging e la verifica del sistema. Se si verifica un errore durante il test del livello k , si può assumere che i livelli da 0 a $k - 1$ siano corretti, isolando il problema.
- **Svantaggi (Efficienza e Design):**
 - **Overhead Prestazionale:** Ogni richiesta di servizio da parte dell'utente deve attraversare molteplici strati per raggiungere l'hardware. Ogni passaggio aggiunge un costo computazionale (overhead), rendendo questi sistemi meno efficienti rispetto a quelli monolitici.
 - **Difficoltà di Definizione:** Stabilire una gerarchia rigorosa è complesso a causa delle dipendenze circolari. Esempio classico: il *driver del disco* (che gestisce l'I/O) deve stare sotto al *gestore della memoria virtuale* (che usa il disco per lo swap)? O viceversa, dato che il driver del disco necessita di buffer in memoria?

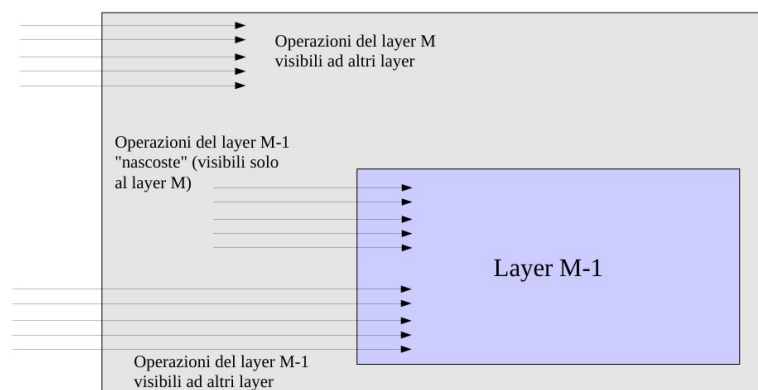


Figura 2.4: **Struttura a Strati.** Le operazioni del Layer $M - 1$ sono "nascoste" e visibili solo al Layer M , garantendo l'incapsulamento.

Evoluzione del Modello a Strati: L'esempio di OS/2

Un esempio più maturo e complesso di architettura a strati è rappresentato dal sistema operativo **OS/2**. In questo modello, la struttura non è solo una pila di livelli che nascondono l'hardware, ma un'organizzazione funzionale pensata per supportare applicazioni complesse.

La struttura si articola nei seguenti livelli (dall'alto verso il basso):

- **Applicazioni:** Il software utente che gira al livello più alto.
- **API (Application Programming Interface):** Le applicazioni non parlano direttamente col kernel, ma si interfacciano con un livello di API e relative estensioni. Questo garantisce stabilità e portabilità del codice.
- **Sottosistemi (Subsystems):** Componenti che implementano le funzionalità specifiche richieste dalle API (es. gestione grafica, gestione finestre).
- **System Kernel:** Il nucleo centrale del sistema operativo. A differenza dei livelli superiori, qui risiedono i meccanismi critici che devono girare con privilegi elevati:
 - Gestione della Memoria;
 - Scheduling della CPU;
 - Gestione dei Dispositivi.
- **Device Drivers:** Moduli specializzati che permettono al Kernel (e in parte ai sottosistemi) di comunicare con l'hardware fisico.

Questa organizzazione "a strati funzionali" anticipa la struttura di molti sistemi moderni, dove la distinzione tra User Mode (Applicazioni, API, Sottosistemi) e Kernel Mode è netta, ma la comunicazione è mediata da interfacce stabili.

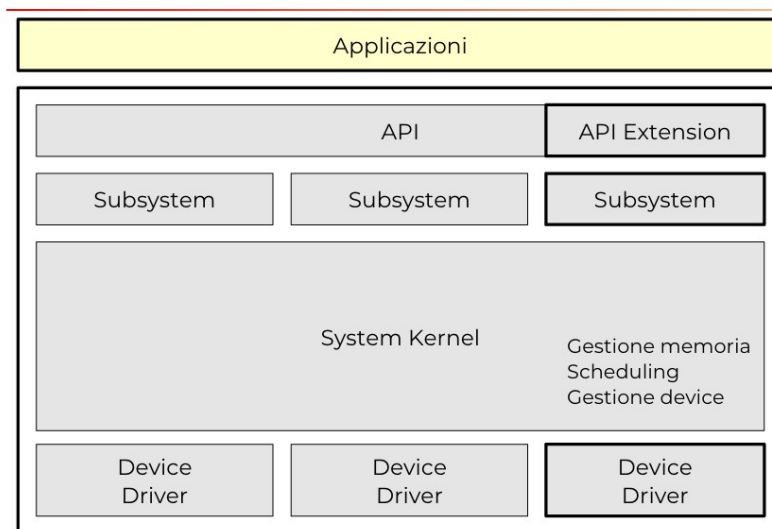


Figura 2.5: **Architettura di OS/2.** Un esempio evoluto di struttura a strati che introduce i concetti di API e Sottosistemi sopra al Kernel di sistema.

Architettura Generale HW/SW (Modello di Myers)

Se l'architettura di OS/2 ci mostra la complessità dei sottosistemi software, il modello proposto da Myers offre una visione strutturale più generale, evidenziando la corrispondenza diretta tra le componenti del Sistema Operativo e l'Hardware sottostante.

In questo schema, la stratificazione è ancora più esplicita:

- **Livello Hardware (Basso):** Qui troviamo le risorse fisiche vere e proprie: l'*Execution Hardware* (CPU), la *Memory Translation* (MMU), il *System Interconnect* (Bus) e i vari *Controller* e dispositivi di I/O.
- **Livello Kernel (Intermedio):** Il Sistema Operativo non è un blocco monolitico indistinto, ma è composto da moduli che gestiscono specificamente le risorse hardware sottostanti:
 - i **Drivers** dialogano con i Controller e i Dispositivi I/O.
 - Il **Memory Manager** controlla l'unità di traduzione della memoria (MMU) e la RAM.
 - Lo **Scheduler** gestisce l'hardware di esecuzione (CPU).
- **Livello Utente (Alto):** Sopra il sistema operativo troviamo le **Librerie** di sistema, che forniscono le funzioni base utilizzate infine dai **Programmi Applicativi**.

Confronto con OS/2: Mentre lo schema di OS/2 enfatizza il ruolo delle API e dei Sottosistemi per la compatibilità e l'interfaccia utente, lo schema di Myers enfatizza il ruolo del Kernel come gestore di risorse hardware. È una visione più "ingegneristica" che mostra come il software di base debba modellarsi perfettamente sull'architettura fisica della macchina (ISA).

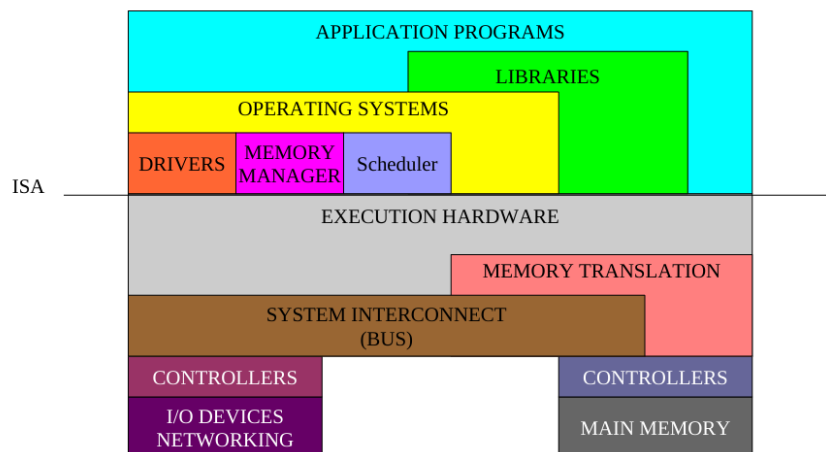


Figura 2.6: **Architettura HW/SW (Myers)**. Schema che illustra la corrispondenza verticale tra le componenti hardware (in basso) e i moduli del kernel responsabili della loro gestione (Driver, Memory Manager, Scheduler), su cui poggiano librerie e applicazioni.

2.3 Politiche e Meccanismi

Un concetto fondamentale di ingegneria del software nei S.O. è la rigida separazione tra *politica* e *meccanismo*.

Definizione – Separazione tra Politica e Meccanismo

- **Politica:** Decide *cosa* deve essere fatto (es. quale processo merita più CPU in questo momento).
- **Meccanismo:** Attua fisicamente la decisione fornendo il *come* (es. il context switch hardware che cambia effettivamente processo).

Separare queste due entità permette di cambiare l'algoritmo decisionale (politica) senza dover riscrivere le funzioni di base (meccanismi), o viceversa. In alcuni S.O. questa separazione non è netta: in MacOS ≤ 9 e Windows 9x, l'interfaccia grafica (politica e meccanismo) era integrata nel kernel per forzare un unico *look and feel*, col rischio che un bug visivo causasse il crash dell'intero sistema.

2.4 Organizzazione del Kernel

Esistono 3 diverse filosofie architetturali con cui i progettisti organizzano il cuore del sistema operativo. La scelta influenza radicalmente l'efficienza, la stabilità e la manutenibilità del sistema.

2.4.1 Kernel Monolitici

È l'approccio classico (e ancora oggi molto diffuso, es. Linux). Il kernel è costituito da un unico, grande aggregato di procedure che vengono eseguite tutte nello stesso spazio di indirizzamento (*Kernel Space*).

- **Struttura:** Sebbene il codice sorgente possa essere modulare, a livello di esecuzione il sistema è un blocco unico. Ogni procedura può chiamare direttamente qualsiasi altra procedura del kernel.
- **Efficienza:** È il punto di forza principale. Poiché tutti i servizi (file system, driver, gestione memoria) girano nello stesso spazio, la comunicazione tra loro è immediata (semplici chiamate a funzione), senza l'overhead di cambi di contesto.
- **Svantaggi:** La mancanza di isolamento interno rende il sistema fragile. Un bug in un driver periferico (che gira in kernel mode) può corrompere le strutture dati del gestore della memoria e mandare in crash l'intero sistema ("kernel panic" o "blue screen").

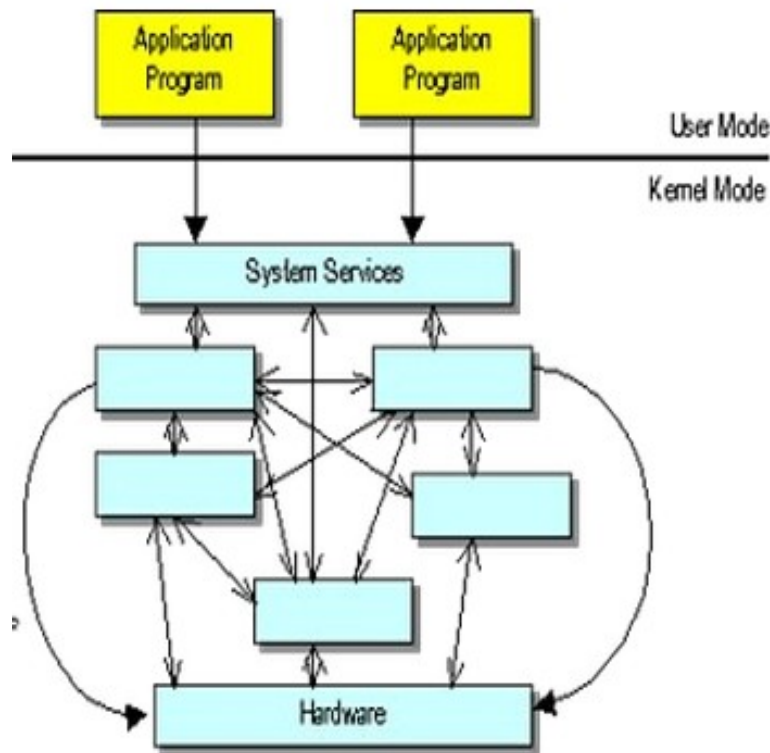


Figura 2.7: **Architettura Monolitica.** Notare come tutti i servizi di sistema (File System, Driver, Scheduling) risiedano nello stesso spazio (Kernel Mode) sotto l'interfaccia delle System Call tutti collegati direttamente tra di loro.

Nota – Evoluzione: I Kernel Modulari

I moderni kernel monolitici (come Linux) hanno mitigato la rigidità iniziale introducendo i **moduli caricabili a runtime**. È possibile caricare driver o funzionalità estese dinamicamente senza ricompilare o riavviare il kernel. Tuttavia, una volta caricati, questi moduli girano comunque in kernel mode, mantenendo i rischi di stabilità del modello monolitico.

2.4.2 Microkernel (Client/Server)

L'architettura a Microkernel nasce dalla necessità di ridurre la complessità crescente dei sistemi a strati tradizionali. L'idea guida è rimuovere dal kernel tutte le componenti non essenziali, implementandole come processi a livello utente (chiamati *server*), lasciando in modalità privilegiata solo un nucleo minimale.

Funzionamento e API

In questo modello, il kernel offre funzionalità ridotte all'osso: gestione minimale di processi e memoria, e meccanismi di comunicazione (IPC). L'API di sistema si basa fondamentalmente su due primitive di message passing:

- `send(message, receiver)`
- `receive(sender)`

Se un'applicazione deve aprire un file (es. `open()`), non esegue una chiamata diretta, ma invia un messaggio al processo server del File System. Il Microkernel agisce da intermediario smistando il messaggio.

Vantaggi del Microkernel

Questa architettura offre benefici strutturali significativi:

- **Robustezza e Tolleranza ai Guasti:** È il vantaggio principale. Se un servizio di sistema (es. il driver del disco o il file system) va in crash, cade solo quel processo utente. Il kernel rimane intatto e il resto del sistema continua a funzionare. Il servizio fallito può essere riavviato dinamicamente senza riavviare la macchina.
- **Estensibilità e Modificabilità:** Per aggiungere un nuovo servizio non serve ricompilare il kernel; basta avviare un nuovo processo server in user space. Questo facilita enormemente lo sviluppo e il debugging.
- **Portabilità:** Essendo il kernel molto piccolo, è più facile portarlo su nuove architetture hardware. I servizi, girando in user space, spesso richiedono solo una ricompilazione.
- **Sicurezza:** È possibile assegnare livelli di sicurezza e privilegi differenti ai vari processi di sistema, riducendo la superficie di attacco rispetto a un kernel monolitico dove tutto gira con privilegi massimi.
- **Supporto ai Sistemi Distribuiti:** Il meccanismo a messaggi si adatta naturalmente alla rete. Per il client è trasparente se il server del file system si trova sulla stessa macchina o su un nodo remoto.

Svantaggi

- **Inefficienza (Overhead):** La comunicazione mediata dal kernel introduce un costo prestazionale. Un'operazione che nel modello monolitico è una singola chiamata a funzione, qui richiede multipli cambi di contesto (Client → Kernel → Server → Kernel → Client) e copie di messaggi in memoria.

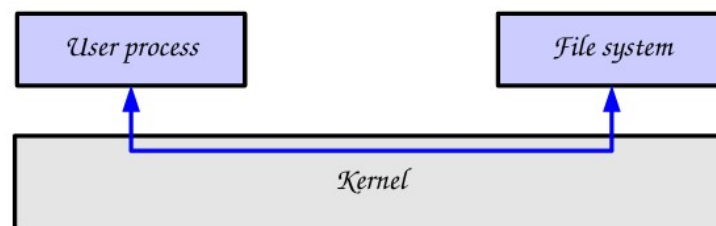


Figura 2.8: **Architettura Microkernel.** Il Kernel è sottile e si occupa solo della comunicazione. I servizi (es. File System) sono processi utente esterni. La freccia mostra il percorso di una richiesta: dal processo utente, attraverso il kernel, fino al server e ritorno. Il kernel quindi gestisce solo i messaggi e l'hW base

Caso di Studio: MINIX

MINIX è un sistema operativo Unix-like scritto da Andrew S. Tanenbaum a scopi didattici, ed è l'esempio classico di architettura a **Microkernel**.

La sua struttura è progettata per massimizzare la modularità e l'affidabilità:

- **Il Kernel:** È ridotto all'osso. Contiene solo il codice per la gestione degli interrupt di basso livello, lo scheduling dei processi e la gestione dei messaggi.
- **I Task (Kernel Threads):** Al livello immediatamente superiore, ma sempre in spazio privilegiato, girano i driver di dispositivo essenziali (Disk, TTY, Clock, Ethernet).
- **I Server (User Space):** Qui risiede la vera particolarità. Componenti che in altri sistemi (come Linux o Windows) sono nel kernel, in MINIX sono normali processi utente isolati:
 - **Memory Manager:** Gestisce la memoria. Decide a chi assegnare la RAM (*politica*), ma per effettuare l'allocazione fisica deve chiedere al kernel di intervenire (*meccanismo*).
 - **File System:** È un processo utente. Se va in crash, non blocca il sistema; il microkernel può semplicemente riavviarlo.

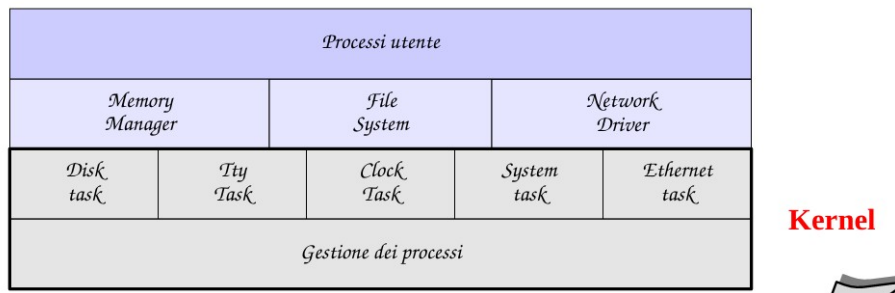


Figura 2.9: **Struttura di MINIX.** Si nota come il "Kernel" sia minuscolo. Servizi critici come il *Memory Manager* e il *File System* girano nello spazio utente (in alto), comunicando tramite messaggi.

Nota – Politiche e Meccanismi in MINIX

MINIX applica rigorosamente la separazione tra *politica* e *meccanismo*. Ad esempio, il Memory Manager è un processo esterno al kernel:

- Decide la *politica* (quanta memoria dare a un processo).
- Non può accedere fisicamente alla memoria degli altri.
- Per attuare la decisione (*meccanismo*), invia una "System Task" al kernel, che esegue l'operazione privilegiata.

2.5 Confronto: Monolitico vs Microkernel

La scelta tra queste due architetture è stata oggetto di storici dibattiti (come la *flamewar* Tanenbaum-Torvalds). Non esiste un "meglio" assoluto, ma dipende dal contesto d'uso.

Nota – Kernel Monolitico: Efficienza Sopra Tutto

Meglio per: Desktop, Server generici, Calcolo ad alte prestazioni.

- *Pro*: La gestione del codice in un unico spazio di indirizzamento è meno complessa dal punto di vista dell'esecuzione. Le prestazioni sono massime grazie all'assenza di overhead di comunicazione.
- *Contro*: Meno robusto. Un errore in un driver può compromettere l'intero sistema.
- *Esempio*: Linux (anche se modulare), Windows (ibrido ma con performance da monolitico).

Nota – Microkernel: Affidabilità Sopra Tutto

Meglio per: Sistemi Embedded, Sistemi Real-Time, Mission-Critical.

- *Pro*: Ideale in contesti dove il fallimento non è un'opzione. La rigorosa separazione garantisce che il sistema resti vivo anche in condizioni critiche.
- *Contro*: Prestazioni inferiori (anche se tecniche moderne hanno ridotto il gap).
- *Esempio*: **QNX** (usato per controllare il braccio robotico dello Space Shuttle o nei sistemi automotive), Minix, L4.

2.5.1 Architetture Alternative

Oltre ai due paradigmi classici (Monolitico vs Microkernel), l'industria e la ricerca hanno sviluppato architetture che cercano di cogliere i pregi di entrambi o di risolvere problemi specifici di performance e flessibilità.

Kernel Ibridi (Macrokernel)

Il Kernel Ibrido rappresenta un compromesso pragmatico tra la purezza architeturale del Microkernel e le prestazioni del Monolitico. L'idea di base è adottare la struttura logica del Microkernel (separando il S.O. in moduli o sottosistemi funzionali), ma di eseguire i componenti critici (come lo stack di rete, il file system o i driver grafici) nello *spazio Kernel* invece che in *User Space*.

- **Funzionamento**: Il sistema è diviso in "sottosistemi" (es. in Windows: sottosistema Win32, sottosistema POSIX, sottosistema OS/2). Tuttavia, per evitare il pesante overhead del *message passing* (il continuo cambio di contesto tra user e kernel mode), questi moduli comunicano tramite meccanismi ad alte prestazioni all'interno del kernel stesso.
- **Esempio (Windows NT/XP/10/11)**: Windows utilizza dei processi server in spazio utente per esporre le API alle applicazioni, ma il cuore esecutivo (l'Executive e i driver) gira interamente in modalità privilegiata per garantire velocità.

Nota – Differenza con il Kernel Monolitico

Sebbene entrambi eseguano gran parte del codice in *Kernel Mode* (e quindi entrambi rischiano il crash totale per un bug in un driver), la differenza è strutturale:

- **Monolitico:** È un blocco unico dove ogni funzione può chiamare qualsiasi altra senza barriere logiche rigide.
- **Ibrido:** È progettato a moduli indipendenti (sottosistemi) che potrebbero teoricamente girare isolati, ma vengono "accorpati" nel kernel solo per motivi di performance. questi inoltre non sono in grado di effettuare il load di moduli dopo la fase di boot

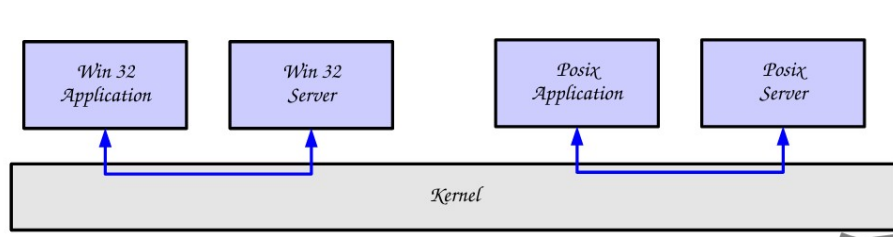


Figura 2.10: **Architettura Ibrida (Windows NT)**. I sottosistemi server gestiscono la personalità del S.O. (API), ma si appoggiano a un Kernel robusto che gestisce l'hardware direttamente.

Exokernel: La Struttura Verticale

L'approccio **Exokernel** è radicale e opposto a quello dei kernel tradizionali (che sono "orizzontali", ovvero offrono astrazioni generiche come "file" o "socket" a tutte le applicazioni). L'Exokernel parte dal presupposto che il S.O. non debba imporre astrazioni, perché spesso le astrazioni generiche penalizzano le prestazioni di applicazioni specifiche (come i database, che vorrebbero gestire il disco a modo loro).

- **Il Kernel:** È piccolissimo e non fornisce astrazioni. Si limita a **multiplexare** le risorse hardware in modo sicuro (es. assegna il settore X del disco o la pagina fisica Y della RAM a un processo), senza nasconderle.
- **LibOS (Library Operating System):** Le astrazioni (file system, gestione memoria virtuale, stack di rete) sono implementate come **librerie** linkate direttamente all'applicazione. Ogni applicazione può portarsi dietro la propria "LibOS" ottimizzata.

Nota – Differenza con il Kernel Monolitico

- **Monolitico:** Il S.O. decide le politiche (es. algoritmo di sostituzione pagine) e le impone a tutti.
- **Exokernel:** Il S.O. garantisce solo la protezione. L'applicazione (tramite la sua libreria) ha il controllo totale sulle decisioni hardware (es. quali pagine di memoria scartare), massimizzando le prestazioni.

AnyKernel

L'architettura **AnyKernel** si concentra sulla sicurezza e sulla rapidità di sviluppo dei driver, permettendo di eseguirli in *User Space* senza dover adottare l'intera

filosofia del Microkernel.

- Consente di caricare driver, file system o stack di rete come normali processi utente.
- Se un driver fallisce, non porta giù l'intero sistema (a differenza del Monolitico), ma può essere riavviato.
- A differenza del Microkernel puro, non impone necessariamente una struttura a messaggi per tutto il sistema, ma si applica in modo selettivo ai componenti che necessitano di isolamento o sviluppo rapido.

2.6 Virtualizzazione e Isolamento

Definizione – Virtuale

Se riusciamo a creare un servizio B differente da A t.c B riesce a sostituire A in maniera efficiente allora B è A virtuale

Oltre all'approccio classico in cui il S.O. divide CPU e memoria creando l'illusione dei processi (multitasking), i sistemi moderni impiegano tecniche di astrazione più avanzate per far coesistere interi ambienti operativi diversi sulla stessa macchina fisica.

2.6.1 Macchine Virtuali (VM)

L'approccio della macchina virtuale sposta l'astrazione a un livello superiore: invece di gestire singoli processi, si simula l'esistenza di un intero calcolatore.

Definizione – Macchina Virtuale (VM)

In senso generale, una **Macchina Virtuale** è un'astrazione software che emula il comportamento di un ambiente di calcolo, isolandolo dall'hardware fisico sottostante.

A seconda del livello in cui avviene questa astrazione, la definizione si declina in due tipologie:

- **System Virtual Machine (SVM)**: Un duplicato efficiente e isolato di un'intera macchina fisica. Un software dedicato (*VMM* o *Hypervisor*) si interpone tra l'hardware e il Sistema Operativo *Guest*, facendogli credere di avere il controllo esclusivo e diretto delle risorse.
- **Process Virtual Machine (PVM)**: Un ambiente di esecuzione virtuale e isolato creato all'interno di un Sistema Operativo *Host*. Maschera l'hardware e l'OS sottostanti per permettere l'esecuzione di un singolo programma (processo) in modo indipendente dalla piattaforma (es. la *Java Virtual Machine*).

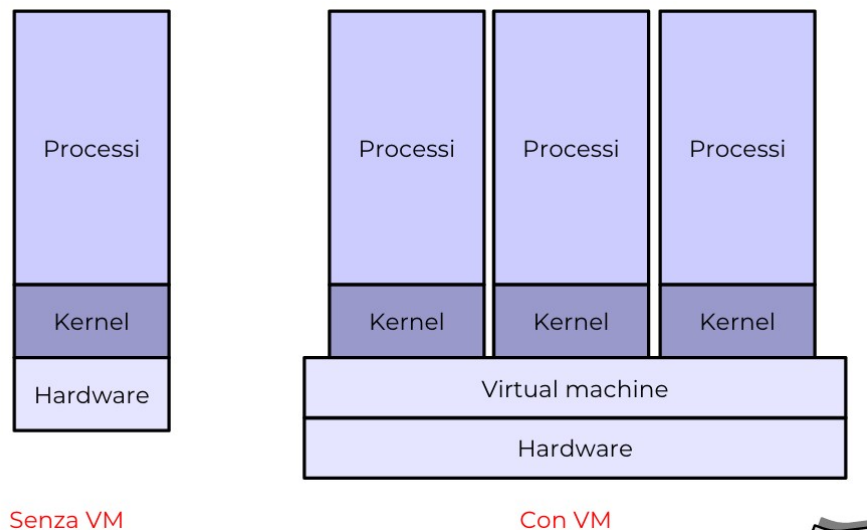


Figura 2.11: **Confronto Architeturale.** A sinistra un sistema classico: il Kernel gestisce direttamente l'hardware. A destra con virtualizzazione: il *Virtualizing Software* (Hypervisor) gestisce l'hardware e ospita molteplici Kernel (Guest OS), ciascuno isolato dagli altri.

Le Interfacce del Calcolatore: ISA e ABI

Per comprendere la virtualizzazione, dobbiamo analizzare dove esattamente "tagliamo" il sistema. Un calcolatore moderno non è un blocco unico, ma una pila di livelli separati da interfacce ben precise.

Guardando l'architettura (figura 2.12), identifichiamo due confini fondamentali:

- **ISA (Instruction Set Architecture):** È il confine basso, tra Hardware e Software. Definisce il vocabolario che la CPU comprende. È divisa in due parti fondamentali:
 - **User ISA:** L'insieme delle istruzioni non privilegiate (calcoli matematici, logica, spostamento dati) che le applicazioni possono eseguire direttamente sull'hardware.
 - **System ISA:** L'insieme delle istruzioni privilegiate (accesso diretto alla memoria, gestione interrupt, I/O) che solo il Sistema Operativo (Kernel Mode) può eseguire.
- **ABI (Application Binary Interface):** È il confine alto, tra le Applicazioni e il Sistema Operativo. Rappresenta l'ambiente totale visto da un programma in esecuzione. L'ABI è composta da:
 - **User ISA:** (Sì, fa parte anche dell'ABI perché l'app deve conoscere il linguaggio macchina).
 - **System Calls:** Il meccanismo con cui l'applicazione chiede servizi al S.O. (es. "leggi file", "apri socket").

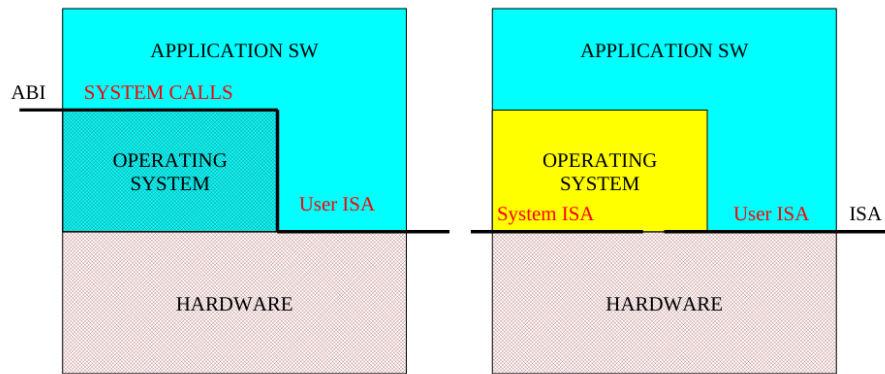


Figura 2.12: **Livelli di Interfaccia (Smith-Nair)**. Si nota come l'**ISA** separi l'Hardware dal Software di base (OS), mentre l'**ABI** separi l'Applicazione dal S.O. L'Applicazione accede all'hardware in due modi: direttamente tramite la *User ISA* o indirettamente tramite le *System Calls* gestite dal S.O.

Nota – Perché questa distinzione è cruciale per le VM?

Questa struttura a strati ci spiega perché esistono due tipi di macchine virtuali:

- Se virtualizziamo l'**ABI** (la linea superiore), otteniamo una **Process VM** (es. Java). L'emulatore traduce le System Call e la User ISA, permettendo al programma di girare su S.O. diversi.
- Se virtualizziamo l'**ISA** (la linea inferiore), otteniamo una **System VM** (es. VirtualBox). L'emulatore ricrea l'hardware fisico (System ISA + User ISA), permettendo di installare un intero S.O. sopra di esso.

1. Process Virtual Machine (PVM)

Queste macchine virtuali (chiamate spesso **Runtime**) operano a livello dell'ABI. Immagina la PVM come un "interprete simultaneo": prende un programma scritto in un formato universale e lo traduce al volo affinché possa girare sul sistema operativo ospite.

- **Obiettivo:** *Portabilità estrema*. Si vuole poter scrivere un programma una volta sola ed eseguirlo su qualsiasi S.O. o hardware.
- **Funzionamento:** Viene lanciato un normale processo (la PVM) sopra il S.O. Host. Questo processo carica l'applicazione "ospite" e le fornisce un ambiente virtuale isolato. Quando l'applicazione termina, la PVM si chiude.

Esempio – Esempi di PVM

- **Java Virtual Machine (JVM):** Permette al codice Java (bytecode) di girare su Linux, Windows o Mac senza modifiche. La JVM maschera le differenze tra i vari sistemi operativi.
- **Rosetta (Apple):** Permette ai nuovi Mac con processori ARM di eseguire vecchi programmi compilati per processori Intel, traducendo le istruzioni in tempo reale.

2. System Virtual Machine (SVM)

Queste sono le macchine virtuali "classiche". Operano a livello dell'ISA. Qui il software di virtualizzazione (**VMM/Hypervisor**) non si limita a far girare un processo, ma costruisce un intero PC finto (CPU finta, Disco finto, RAM finta).

- **Obiettivo:** *Consolidamento e Isolamento*. Far girare più Sistemi Operativi completi contemporaneamente sulla stessa macchina fisica.
- **Funzionamento:** Il VMM inganna il Sistema Operativo Guest (es. Windows dentro Linux). Il Guest crede di essere l'unico padrone dell'hardware ("Bare Metal"), ma in realtà ogni volta che prova a toccare l'hardware (es. istruzioni privilegiate), il VMM intercetta l'operazione e la gestisce per lui.
- **Esempi:** *VMware, VirtualBox, QEMU/KVM*.

Approfondimento: La Transizione verso la Virtualizzazione

Le figure mostrano come cambia l'architettura di un calcolatore quando si introduce la virtualizzazione. La differenza chiave sta nel *dove* viene inserito lo strato di software di virtualizzazione (*Virtualizing SW*).

Caso 1: PVM (Process Virtual Machine) Immagina di inserire un "interprete" **sopra** il Sistema Operativo esistente (proprio sotto l'applicazione).

- **Prima:** L'applicazione invia istruzioni direttamente al S.O. (es. "apri finestra").
- **Dopo:** L'applicazione invia istruzioni alla PVM (es. JVM). La PVM traduce queste istruzioni per il S.O. ospitante.
- **Effetto:** L'applicazione è portabile ovunque, ma vede ancora lo stesso S.O. sottostante per i servizi di base.

Caso 2: SVM (System Virtual Machine) Immagina di inserire un "simulatore di hardware" **sotto** il Sistema Operativo (a diretto contatto con l'hardware).

- **Prima:** Il S.O. invia comandi elettrici direttamente alla CPU e al Disco.
- **Dopo:** Il S.O. (detto Guest) invia comandi al VMM (Hypervisor). Il VMM finge di essere un disco o una CPU e gestisce la richiesta sull'hardware reale.
- **Effetto:** Il S.O. è completamente ingannato e isolato. Possiamo eseguire più S.O. (es. Windows e Linux) contemporaneamente perché il VMM spartisce l'hardware reale tra di loro.

Classical Theory of VM

Process / Machine VM (Smith-Nail 2005)

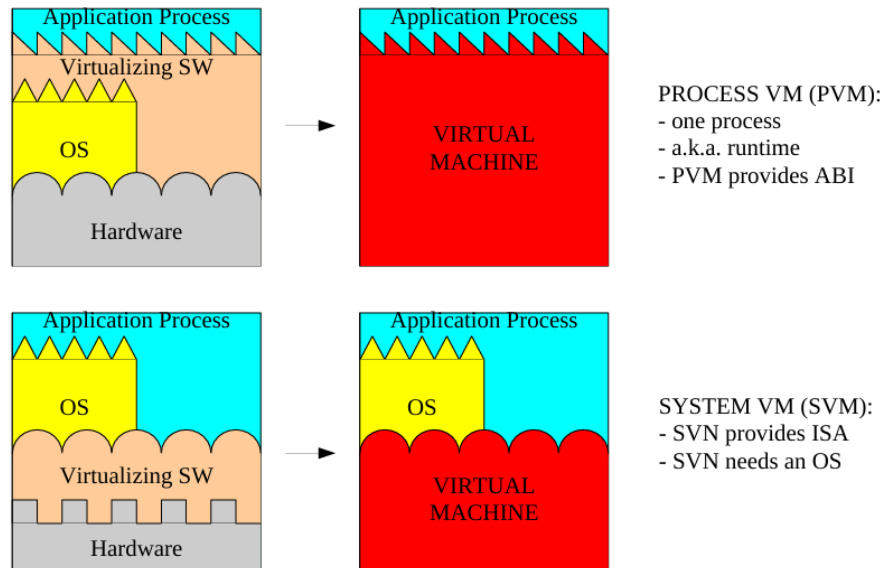


Figura 2.13: **Confronto Visivo PVM vs SVM.** *alto (PVM)*: Il livello di virtualizzazione (arancione) è inserito tra l'OS e l'Applicazione. L'OS rimane unico e invariato. *basso (SVM)*: Il livello di virtualizzazione è inserito tra l'Hardware e l'OS. Questo permette di avere un OS "ospite" (Giallo) che gira sopra l'hardware virtualizzato.

Vantaggi e Svantaggi delle System VM

L'utilizzo delle macchine virtuali di sistema comporta benefici strutturali ma anche costi in termini di risorse.

Nota – Vantaggi

- **Coesistenza di S.O. Differenti:** È possibile far girare sistemi diversi (es. Linux e Windows) sulla stessa macchina fisica. Utile per testare nuove release software senza formattare il computer.
- **Sicurezza e Isolamento (Sandbox):** Ogni VM è isolata. Se un virus infetta il Guest OS, o se il sistema va in crash, la macchina Host e le altre VM non subiscono danni.
- **Supporto Legacy e Sicurezza:** È possibile eseguire vecchi sistemi operativi *monotask* (es. MS-DOS) all'interno di un sistema *multitask* moderno. Il sistema monotask gira in un ambiente protetto: se si blocca, si chiude solo la finestra della VM, non l'intero computer.
- **Emulazione Hardware:** È possibile simulare architetture hardware diverse da quella fisica (es. sviluppare e testare software per processori ARM o MIPS usando un normale PC con processore Intel).

Nota – Svantaggi

- **Inefficienza:** L'emulazione via software è lenta. Ogni istruzione privilegiata del Guest deve essere intercettata e simulata dal VMM, introducendo un overhead significativo. (Nota: le moderne CPU hanno estensioni hardware come Intel VT-x per ridurre questo problema).
- **Difficoltà nella condivisione delle risorse:** Le risorse (RAM, spazio disco) vengono spesso allocate in modo statico alla VM. Se assegno 8GB di RAM a una VM, questi vengono tolti all'Host, anche se la VM non li sta usando tutti in quel momento. Condividere dinamicamente file o memoria tra Host e Guest è complesso e richiede driver aggiuntivi.

2.7 Namespace e Container (Virtualizzazione a Livello S.O.)

Abbiamo visto che le Macchine Virtuali di sistema (SVM) risolvono il problema dell'isolamento emulando hardware fisico, con un costo alto: ogni VM deve avviare un intero sistema operativo Guest, consumando GB di RAM e decine di secondi al boot.

Ma spesso *non abbiamo bisogno di tutto questo*. Se vogliamo solo isolare un'applicazione web, un microservizio o un ambiente di sviluppo, emulare CPU e disco è uno spreco enorme. Linux ha sviluppato due meccanismi kernel nativi, **Namespace** e **Cgroups**, che permettono di ottenere un isolamento sufficiente senza emulare nulla.

2.7.1 Il Problema: Perché Isolare i Processi?

In un sistema Linux classico, tutti i processi condividono un'unica visione globale delle risorse:

- Tutti vedono la stessa lista di processi (albero PID globale).
- Tutti condividono lo stesso stack di rete (interfacce, routing, porte).
- Tutti vedono lo stesso file system montato.
- Un processo con i privilegi giusti può interferire con qualsiasi altro processo.

Questo va bene per applicazioni tradizionali, ma crea problemi gravi quando vogliamo eseguire più servizi indipendenti sulla stessa macchina: un'applicazione buggy potrebbe leggere la memoria di un'altra, due servizi potrebbero collidere sulla stessa porta di rete, o un processo malevolo potrebbe ispezionare l'albero dei processi dell'host.

Definizione – Namespace

Un **Namespace** è una funzionalità del kernel Linux che crea una *visione privata e isolata* di una specifica risorsa globale per un gruppo di processi. I processi che appartengono a un namespace vedono solo gli oggetti di quel namespace (i *loro* processi, la *loro* rete, il *loro* file system). Dal punto di vista del processo, sembra di essere l'unico sul sistema. In realtà, il kernel gestisce più namespace in parallelo, mantenendo tabelle di mapping separate.

Nota – Analogia Chiave

Pensa ai namespace come a **occhiali con filtri colorati**. Ogni gruppo di processi indossa un paio di occhiali diversi: vedono lo stesso kernel (la stessa realtà sottostante), ma la loro visione filtrata è completamente diversa. Un processo con occhiali rossi non vede i processi con occhiali blu, e viceversa. Il kernel è l'unico che vede tutto senza filtri.

2.7.2 I Tipi di Namespace in Linux

Il kernel Linux implementa diversi tipi di namespace, ognuno responsabile dell'isolamento di una risorsa specifica.

PID Namespace — Isolamento dei Processi

Il **PID Namespace** virtualizza l'albero dei processi. All'interno del namespace, il primo processo avviato riceve PID 1, esattamente come `init/systemd` nella macchina reale. Tutti i processi figli avranno PID progressivi.

Esempio – PID Namespace in pratica

Immagina che il sistema host abbia un processo con PID 4500, che è il server Nginx di un container. Fuori dal namespace (lato host) il kernel vede:

- PID 4500 → nginx (processo host normale)

Ma all'interno del PID Namespace del container, lo stesso processo viene visto come:

- PID 1 → nginx (primo processo del container, come se fosse init)

Il mapping è gestito interamente dal kernel. Il processo non sa del suo "doppio PID".

Perché è importante? Se un processo dentro il namespace manda `kill -9 1` (il comando "uccidi init"), distrugge se stesso — non il sistema host. L'isolamento protegge la macchina reale.

NET Namespace — Isolamento della Rete

Il **NET Namespace** virtualizza l'intero stack di rete. Ogni namespace possiede:

- Le proprie interfacce di rete (es. una `eth0` virtuale privata).
- Le proprie tabelle di routing.
- Le proprie regole `iptables`/firewall.
- La propria tabella di porte aperte (socket).

Esempio – NET Namespace in pratica

Senza namespace di rete, due server web non possono entrambi ascoltare sulla porta 80 della stessa macchina. Con NET Namespace, il container A ha la *sua* porta 80, il container B ha la *sua* porta 80, e il kernel mappa queste porte a porte diverse sulla rete fisica (es. `host:8080 → containerA:80`, `host:8081 → containerB:80`). È come avere due case ognuna con il suo citofono, ma il postino (kernel) si occupa di instradare correttamente la posta.

MNT Namespace — Isolamento del File System

Il **MNT (Mount) Namespace** isola i punti di montaggio del file system. I processi dentro il namespace vedono una gerarchia di directory completamente diversa da quella dell'host.

È su questo namespace che si basa il meccanismo di **chroot esteso**: il container vede una propria `/`, con il proprio `/bin`, `/etc`, `/var` — completamente indipendente dall'host.

Nota – Implicazione importante

Poiché ogni container ha il suo MNT Namespace, è possibile che un container Ubuntu e un container Alpine girino *sulla stessa macchina Debian*: ognuno vede la propria distribuzione nel file system, anche se condividono lo stesso kernel Linux dell'host. La scelta della distribuzione riguarda solo i *file utente* (librerie, binari), non il kernel.

USER Namespace — Isolamento degli Utenti

Il **USER Namespace** gestisce mapping separati degli UID (User ID) e GID (Group ID).

Esempio – USER Namespace in pratica

Un processo può essere **root** (UID 0) all'interno del suo namespace, ma essere mappato a un utente normale senza privilegi (es. UID 1000) sull'host. Questo è fondamentale per la sicurezza: il processo crede di avere poteri illimitati, ma in realtà non può fare nulla di dannoso fuori dal container.

UTS, IPC Namespace

- **UTS Namespace**: Consente di avere hostname e dominio separati per ogni container. Ogni container può "credere" di chiamarsi **webserver-prod** o **db-staging** senza influenzare il nome del sistema host.
- **IPC Namespace**: Isola le risorse di comunicazione inter-processo: code di messaggi POSIX, semafori System V, memoria condivisa. Un container non può accedere alla memoria condivisa di un altro.

2.7.3 Le System Call dei Namespace**Nota – Come si creano i Namespace: le System Call**

La creazione e la gestione dei namespace avviene tramite tre system call dedicate:

- **clone(flags)**: Crea un nuovo processo figlio. Attraverso flag specifici (es. **CLONE_NEWPID**, **CLONE_NEWNET**, **CLONE_NEWNS**) si istruisce il kernel affinché il figlio venga creato *dentro nuovi namespace separati* dal genitore. Questa è la chiamata usata internamente da Docker quando avvia un container.
- **unshare(flags)**: Il processo chiamante si *stacca* dai namespace correnti e si sposta in nuovi namespace appena creati. Utile per creare ambienti isolati senza fork. Il comando shell **unshare -p /bin/bash** avvia una bash in un nuovo PID namespace.
- **setns(fd, nstype)**: Permette a un processo di *entrare* in un namespace già esistente, identificato dal suo file descriptor. È la system call usata da **docker exec**, che deve "agganciarsi" ai namespace di un container già in esecuzione per eseguire un comando al suo interno.

Ogni namespace creato è rappresentato nel filesystem come un file speciale in **/proc/\$PID/ns/**. Finché esiste almeno un processo nel namespace (o il file è tenuto aperto), il name-

space rimane attivo. Quando tutti i processi escono, il namespace viene distrutto e le risorse liberate.

2.7.4 Cgroups: Limitazione delle Risorse

I namespace risolvono l'*isolamento della visibilità* (cosa un processo può vedere), ma non risolvono il problema dell'*uso delle risorse*: un processo isolato potrebbe comunque consumare tutta la CPU o tutta la RAM, affamando gli altri.

Questo è il compito dei **Cgroups (Control Groups)**, l'altro pilastro della containerizzazione.

Definizione – Cgroups (Control Groups)

I **Cgroups** sono una funzionalità del kernel Linux che permette di organizzare i processi in *gruppi gerarchici* e di applicare a ciascun gruppo **limiti rigidi** sull'utilizzo delle risorse hardware:

- **CPU**: Percentuale massima di tempo CPU utilizzabile.
- **Memoria**: Quantità massima di RAM (e swap) allocabile.
- **I/O**: Velocità massima di lettura/scrittura su disco.
- **Rete**: Banda massima utilizzabile.

Quando un gruppo supera il limite di memoria, il kernel uccide i suoi processi (OOM Killer). Quando supera il limite CPU, i processi vengono throttled (rallentati).

2.7.5 Container: Namespace + Cgroups

Definizione – Container

Un **Container** è un *gruppo di processi* che il kernel tratta come un ambiente isolato e limitato, ottenuto combinando:

1. **Namespace**: Per l'isolamento della visibilità (ogni container ha la sua "bolla" privata di rete, file system (fornito dall'immagine docker), processi, utenti).
2. **Cgroups**: Per la limitazione delle risorse (ogni container ha quote rigide di CPU, RAM, I/O).

Tutti i container condividono lo stesso Kernel dell'host. Non c'è emulazione hardware, non c'è boot di un S.O. Guest. Un container è, a tutti gli effetti, un insieme di processi normali con una visione ristretta del sistema.

Esempio – Dai Namespace ai Container: un Esempio Step-by-Step

Supponi di voler creare manualmente un container minimalista che esegua una bash isolata:

1. **Crea i namespace**:
`unshare -pid -net -mount -uts -ipc -fork /bin/bash`
Questo crea una nuova bash in nuovi namespace PID, NET, MNT, UTS, IPC.
2. **Isola il file system (chroot)**:
`chroot /path/to/ubuntu-rootfs /bin/bash`

La bash ora vede solo il file system Ubuntu nella directory `ubuntu-rootfs/`.

3. Applica limiti con cgroups:

Scrivi il PID della bash nel cgroup opportuno in `/sys/fs/cgroup/`.

Hai appena ricreato, a mano, il 90% di quello che fa Docker. Docker non è magia: è un sistema di gestione che automatizza esattamente questi passi.

2.7.6 Confronto: Macchine Virtuali vs Container

Nota – Differenze Fondamentali

Caratteristica	Macchina Virtuale	Container
Livello astrazione	Hardware (ISA)	S.O. (Namespace/Cgroups)
Kernel	Proprio (Guest OS)	Condiviso con l'host
Avvio	Decine di secondi (boot)	Frazioni di secondo
Peso	GB (immagine disco)	MB (solo file utente)
Isolamento	Fortissimo (hardware separato)	Buono (namespace kernel)
Portabilità	Media (dipende dall'hypervisor)	Alta (stessa immagine ovunque)
Overhead	Alto (emulazione hardware)	Minimo (processi nativi)

Quando usare cosa?

- **VM:** Quando serve massimo isolamento (ambienti di sicurezza, esecuzione di S.O. diversi), o quando il workload richiede un kernel diverso da quello dell'host.
- **Container:** Quando servono scalabilità, velocità di deploy e coerenza dell'ambiente di sviluppo/produzione. Ideali per microservizi e applicazioni cloud-native.

Non sono mutuamente esclusivi: in produzione è comune avere container che girano *dentro* VM (es. su AWS, ogni istanza EC2 è una VM, e sopra ci girano i container Docker).

2.7.7 Docker: il Runtime di Container (EXTRA)

Docker è lo strumento più diffuso per creare, distribuire ed eseguire container. È importante capire che Docker non è una tecnologia di virtualizzazione nuova: è uno *strato di gestione* sopra Namespace e Cgroups, che li rende usabili in modo pratico.

Architettura di Docker

Docker è composto da più componenti che interagiscono:

- **Docker Daemon (dockerd)**: Il processo server che gira in background sull'host. Gestisce tutta la vita dei container: creazione, avvio, stop, networking, volumi. Parla con il kernel tramite le system call dei namespace e cgroups.
- **Docker CLI (docker)**: Il client a riga di comando con cui l'utente interagisce. I comandi come `docker run` o `docker ps` sono richieste inviate al Daemon tramite una REST API.
- **containerd**: Il runtime di livello inferiore su cui si appoggia `dockerd`. Gestisce il ciclo di vita dei container a basso livello (pull delle immagini, creazione dei namespace, esecuzione dei processi). È un progetto separato donato alla CNCF.
- **runc**: Il runtime di livello ancora più basso. È il componente che effettivamente chiama `clone()` e configura i cgroups. Implementa le specifiche OCI (Open Container Initiative).

Immagini Docker e Layered File System

Definizione – Immagine Docker

Un'**Immagine Docker** è un file system read-only stratificato (*layered*), che contiene tutto il necessario per eseguire un'applicazione: sistema operativo base (solo i file utente, non il kernel), librerie, dipendenze, configurazioni e codice dell'applicazione.

Un **Container** è un'immagine Docker a cui viene aggiunto un livello scrivibile in cima (*copy-on-write layer*): le modifiche fatte durante l'esecuzione vanno su questo strato temporaneo e vengono perse quando il container viene eliminato (a meno di usare i *Volumi*).

Il layered file system è implementato tramite **OverlayFS**, un filesystem Linux che permette di sovrapporre più directory (layer) e presentarle come una sola. Questo approccio ha due vantaggi fondamentali:

- **Riutilizzo**: Se 10 container usano la stessa immagine base Ubuntu, il layer base esiste una volta sola su disco. Solo il layer scrivibile è duplicato.
- **Velocità di distribuzione**: Le immagini si trasferiscono layer per layer. Se condividi un nuovo container con una base già scaricata, scarichi solo i layer nuovi (delta).

Nota – Immagine vs Container: Qual è la reale differenza?

il **Container** è l'istanza in esecuzione dell'**Immagine**.

- **L'Immagine (Il Progetto):** È un'entità statica, immutabile e in sola lettura (*read-only*). Corrisponde, nell'analogia della programmazione ad oggetti, a una *Classe*.
- **Il Container (L'Esecuzione):** È l'entità dinamica. Corrisponde a un *Oggetto* istanziato da quella classe.

A livello di File System, quando si avvia un container, Docker non copia l'intera immagine. Si limita a prendere i livelli in sola lettura dell'immagine e vi sovrappone un nuovo livello scrivibile (*Thin Writable Layer*). Tutte le operazioni di scrittura, modifica o cancellazione effettuate dal software in esecuzione avvengono **esclusivamente** in questo strato superficiale, lasciando l'immagine sottostante perfettamente intatta e riutilizzabile contemporaneamente da altri container.

A cosa serve l'Immagine?

Se il container è solo un "processo isolato e ingannato", a cosa serve l'Immagine Docker?

Per rendere l'illusione perfetta, questo processo isolato ha bisogno di un suo **finto disco fisso** in cui navigare, trovare i suoi file, le sue librerie e i suoi comandi di base (come `ls`, `cd`, `apt`). **L'Immagine è esattamente questo: il finto disco fisso (il File System) che viene fornito al processo isolato.**

Definizione – Il Container (Definizione Completa)

Un **Container** non è un oggetto fisico e non è un file. È un *ambiente di esecuzione logico*. È il risultato dell'unione di tre elementi che lavorano simultaneamente:

1. Un **Processo in esecuzione** (il software vero e proprio che sta lavorando nella RAM).
2. Un set di **Regole di Isolamento** (Namespace e Cgroups imposti dal Kernel).
3. Un **File System privato** (fornito dall'Immagine Docker).

Quando il processo viene terminato, l'illusione svanisce e il container smette di esistere, mentre l'Immagine (il disco finto) rimane salvata sul PC pronta per essere usata di nuovo.

Esempio – L'Analogo del The Truman Show

Per capire la differenza tra Container e Immagine, pensa al film *The Truman Show*:

- **L'Immagine Docker** è l'enorme set televisivo costruito finto: le case di cartapesta, le strade, i finti prodotti nel supermercato (i file, le cartelle, le librerie). È un costrutto statico.
- **I Namespace e i Cgroups** sono la gigantesca cupola che isola il set dal mondo reale e le regole che impediscono a Truman di uscire.
- **Il Container** è *l'intera trasmissione in diretta*: Truman (il processo)

che vive la sua vita dentro la cupola (isolamento), interagendo con il set di cartapesta (l'immagine), convinto che quello sia il mondo intero (virtualizzazione a livello S.O.).

Il Dockerfile: Costruire un'Immagine

Un **Dockerfile** è un file di testo che descrive, passo dopo passo, come costruire un'immagine Docker. Ogni istruzione crea un nuovo layer nell'immagine finale.

Esempio – Dockerfile per una Web App Python

Listing 2.1: Dockerfile esempio

```
# Layer 1: immagine base (sistema Ubuntu + Python
preinstallato)
FROM python:3.11-slim

# Layer 2: imposta la directory di lavoro nel
container
WORKDIR /app

# Layer 3: copia il file delle dipendenze e le
installa
COPY requirements.txt .
RUN pip install -r requirements.txt

# Layer 4: copia il codice dell'applicazione
COPY . .

# Comando eseguito quando il container viene avviato
CMD ["python", "app.py"]
```

Il comando `docker build -t myapp .` legge questo file e produce un'immagine con 4 layer sovrapposti. Modificare solo il codice dell'app (Layer 4) non ricostruisce i layer precedenti, risparmiando tempo.

Aggiornamento del Software nei Container

Problema: l'Immutabilità dei Container I container Docker sono progettati per essere **immutabili**: un container non dovrebbe mai essere aggiornato in-place durante l'esecuzione. Se l'applicazione ha un bug o una nuova versione, il flusso corretto è:

1. **Ferma** il container vecchio.
2. **Costruisci** una nuova immagine con la versione aggiornata.
3. **Avvia** un nuovo container dalla nuova immagine.

Questo significa che il software (librerie, dipendenze, binari) è *incluso dentro ogni immagine*. Se hai 50 container che usano la stessa libreria vulnerabile, devi ricostruire e rilanciare tutti e 50. È un costo operativo, ma garantisce riproducibilità e tracciabilità delle versioni.

2.8 Progettazione e System Generation

La progettazione di un sistema operativo non è un processo univoco, ma dipende strettamente dai **vincoli (constraints)** e dagli **obiettivi**:

- **Vincoli Hardware:** Su che macchina deve girare? (Microcontrollore, PC, Mainframe).
- **Vincoli Applicativi:** Qual è lo scopo? (Sistema Batch, Time-sharing, Real-time, Distribuito).

Poiché i sistemi operativi moderni (come UNIX, Linux, Windows) sono progettati per essere **portabili** su architetture molto diverse, è necessario un meccanismo per adattare il software generico allo specifico hardware disponibile. Questo processo è detto **System Generation (SYSGEN)**.

2.8.1 Parametri di Configurazione

Durante la generazione del sistema, vengono definiti i parametri vitali che modellano il S.O. sulla macchina target:

- **Tipo di CPU:** Architettura (x86, ARM), set di istruzioni, numero di core.
- **Memoria:** Quantità di memoria centrale disponibile.
- **Periferiche:** Quali dispositivi sono collegati e quali driver includere.
- **Parametri Numerici (Tuning):**
 - Numero massimo di utenti concorrenti.
 - Numero massimo di processi attivi.
 - Dimensioni dei buffer di I/O e delle tabelle di sistema.

2.8.2 Metodi di Generazione

Esistono due approcci principali per adattare il S.O.:

1. **Ricompilazione del Kernel (Statico):** È il metodo tradizionale (tipico dei sistemi UNIX/Linux puri). Si modifica il file di configurazione con i parametri desiderati e si ricompila l'intero codice sorgente del kernel. Il risultato è un file binario monolitico "cucito su misura" per quella macchina.
2. **Moduli Dinamici (Loadable Modules):** È l'approccio moderno. Il kernel base è generico, ma è capace di rilevare l'hardware e caricare i driver necessari al volo durante il *boot* o a sistema avviato.
 - **Linux:** Moduli `.ko` (Kernel Object).
 - **macOS:** Kernel Extensions (`.kext`).
 - **Windows:** Driver `.sys`.